NTT(Number Theoretic Transform)

1 PRELIMINARIES

1.1 Notations and Definitions

- $\mathbb{Z}$: the ring of rational integers
- n, q: some positive integers
- $\mathbb{Z}_q \cong \{0, 1, \ \ldots, \ q-1\}$
- $x' \equiv x \pmod{q}$: $q \mid (x' - x)$
- $x' = x \pmod{q}$: x' is the unique element in $\mathbb{Z}_q$ satisfying $x' \equiv x \pmod{q}$
- $\mathbf{a}$, $a(x)$: polynomials
- $\mathbf{r} = \mathbf{a} \mod \mathbf{b}$: r is the polynomial remainder of a divided by b
- For any $n_1, n_2 \in \mathbb{Z}$, $n_1 > n_2$ and any a_i, $\sum_{j=n_1}^{n_2} a_i = 0$
- $\circ$: point-wise multiplication

Def) Bitreversal

Let n be a power of two, and b be a non-negative integer satisfying $b < n$.

The bitreversal of b with respect to n is defined as

$$\mathrm{brv}_n(b_{\log_{n-1}} 2^{\log n - 1} + \cdots + b_1 2 + b_0) = b_0 2^{\log n - 1} + \cdots + b_{\log_{n-2}} 2 + b_{\log_{n-1}},$$

where b_i is the i-th bit of the binary expansion of b.

Def) Primitive and Principal Root of Unity

The elements of the set $U_k = \{\psi \in \mathbb{C} \mid \psi^k = 1\}$ are called the k-th roots of unity.

Let R be a commutative ring with multiplicative identity 1, k be a positive integer, and $\psi(\neq 1)$ be an element in R.

Define ψ is the primitive k-th root of unity in $R \iff \psi^k = 1$, and $\psi^i \neq 1, \ i = 1, 2, \ \ldots, \ k-1$.
(i.e., k is the least positive integer s.t. $\psi^k = 1$)

Define ψ is the principal k-th root of unity in $R \iff \psi^k = 1$, and $\sum_{j=0}^{k-1} \psi^{jl} = 0, \ l = 1, 2, \ \ldots, \ k-1$.

Notice that primitive and principal k-th root of unity coincide if $R = \mathbb{Z}_q$ where q is a prime number.

- Coincide if $R = \mathbb{Z}_q$

  1. Primitive ⇒ Principal

     $\mathbb{Z}_q$의 primitive k-th root of unity ψ가 존재하면, ψ는 $\mathbb{Z}_q$의 principal k-th root of unity이다.

     pf) ψ가 primitive k-th root of unity이므로

$\psi^k \equiv 1 \bmod q$이고, 모든 $i = 1, 2, \ldots, k-1$에 대하여 $\psi^i \neq 1$이다.

즉, 모든 $l = 1, 2, \ldots, k-1$에 대해 $\psi^l - 1 \neq 0$이므로

$$\sum_{j=0}^{k-1} \psi^{jl} = \frac{(\psi^l)^k - 1}{\psi^l - 1} = \frac{(\psi^k)^l - 1}{\psi^l - 1} = \frac{(1)^k - 1}{\psi^l - 1} = 0$$

이다.

$\psi^k \equiv 1 \bmod q$이고, 모든 $l = 1, 2, \ldots, k-1$에 대해 $\sum_{j=0}^{k-1} \psi^{jl} = 0$이므로

ψ는 principal k-th root of unity이다.

2. Principal ⇒ Primitive

   $\mathbb{Z}_q$의 principal k-th root of unity ψ가 존재하면, ψ는 $\mathbb{Z}_q$의 primitive k-th root of unity이다.

   pf) ψ가 $\mathbb{Z}_q$의 principal k-th root of unity일 때,

   어떤 $l \in \{1, 2, \ldots, k-1\}$에 대하여 $\psi^l = 1$이라고 가정하자.

   이 l에 대해

   $$\sum_{j=0}^{k-1} \psi^{jl} = (\psi^l)^0 + (\psi^l)^1 + \cdots + (\psi^l)^{k-1} = 1 + 1 + \cdots + 1 = k \neq 0 \pmod q$$

   이다. ($k \mid (q-1)$이어야 하므로 $k < q$) 이는 ψ가 principal k-th root of unity라는 가정에 모순이므로 ψ는 모든 $l = 1, 2, \ldots, k-1$에 대해 $\psi^j \neq 1$이다. 모든 $j = 1, 2, \ldots, k-1$에 대하여 $\psi^j \neq 1$이고, $\psi^k \equiv 1 \bmod q,\ \psi \neq 1$이므로 ψ는 principal k-th root of unity이다.

- Example

  – $\mathbb{Z}_5$에서 a^i Table

$\psi \setminus i$	0	1	2	3	4
1	1	1	1	1	1
2	1	2	4	8=3	**6=1**
3	1	3	9=4	12=2	**6=1**
4	1	4	16=1	4	16=1

  – 2, 3 are primitive 4-th roots of unity in $\mathbb{Z}_5$

ψ	$\sum_{j=0}^{3} a^j$	$\sum_{j=0}^{3} a^{2j}$	$\sum_{j=0}^{3} a^{3j}$
1	4	4	4
2	0	0	0
3	0	0	0
4	0	4	0

  2, 3 are principal 4-th roots of unity in $\mathbb{Z}_5$

  → primitive and principal 4-th roots of unity coincide in $\mathbb{Z}_5$

1.2 Polynomial Rings

Let $\mathbb{Z}[x]$ and $\mathbb{Z}_q[x]$ be the polynomial rings over $\mathbb{Z}$ and $\mathbb{Z}_q$ respectively, with corresponding quotient rings $\mathbb{Z}[x]/\langle\phi(x)\rangle$ and $\mathbb{Z}_q[x]/\langle\phi(x)\rangle$, where $\phi(x)$ is a polynomial with integer coefficients.

Preliminary

- Def) zero of $f(x)$

  Let F be a subfield of a field E, and let α be an element of E.
  Let $f(x) = a_0 + a_1 x + \cdots + a_n x^n$ be in $F[x]$, and let $\phi_\alpha : F[x] \to E$ be the evaluation homomorphism.

  Let $f(\alpha)$ denote

$$\phi_\alpha(f(x)) = a_0 + a_1\alpha + \cdots + a_n\alpha^n.$$

  If $f(\alpha) = 0$, then α is a zero of $f(x)$.

- Def) factor of $f(x)$

  An element $a \in F$ is a zero of $f(x) \in F[x]$ if and only if $x - a$ is a factor of $f(x)$ in $F[x]$.

- Thm) Eisenstein Criterion

  Let $q \in \mathbb{Z}$ be a prime.

  Suppose that $f(x) = a_n x^n + \cdots + a_0$ is in $\mathbb{Z}[x]$, and $a_n \neq 0 \pmod q$, but $a_i = 0 \pmod q$ for all $i < n$, with $a_0 \neq 0 \pmod{q^2}$.

  Then $f(x)$ is irreducible over $\mathbb{Q}$.

Especially, $\phi(x)$ is chosen to be a cyclotomic polynomial.

Preliminary

- Cor) The polynomial

$$\phi_p(x) = \frac{x^p - 1}{x - 1} = x^{p-1} + \cdots + x + 1$$

  is irreducible over $\mathbb{Q}$ for any prime p.

- The polynomial $\phi_p(x)$ is the p-th cyclotomic polynomial.

  n-th cyclotomic polynomial $\phi_n(x)$는 1의 primitive n-th roots of unity들만을 근으로 갖는 다항식이다.

$$\phi_n(x) = \prod_{1 \le k < n, \gcd(k,n)=1} (x - e^{2\pi k i/n})$$

  p가 소수일 때, 방정식 $x^p - 1 = 0$은 복소평면 위에서 총 p개의 해를 가진다. p개의 근을 ζ_k라고 하면

$$\zeta_k = e^{2\pi k i/n} \quad (k = 0, 1, \ldots, p-1)$$

  이고, $k = 0$일 때 $\zeta_0 = 1$이다. 이는 order가 1이므로 primitive p-th roots of unity가 아니다.

  $k \neq 0$인 경우, p가 prime이므로, 1보다 크고 p보다 작은 모든 k는 $\gcd(k, p) = 1$을 만족한다. 따라서, 1을 제외한 나머지 $p - 1$개의 근은 모두 order=p인 primitive roots of unity이다.

  따라서

$$x^p - 1 = \prod_{k=0}^{p-1}(x - \zeta_k) = (x - 1)\prod_{k=1}^{p-1}(x - \zeta_k)$$

  이고, 정의에 따라

$$\prod_{k=1}^{p-1}(x - \zeta_k) = (x^p - 1)/(x - 1)$$

  은 cyclotomic polynomial이 된다.

If $\deg \phi(x) = n$ holds, the element in $\mathbb{Z}_q[x]/\langle\phi(x)\rangle$, for example, $\mathbf{a}$, can be represented in the form of $\mathbf{a} = \sum_{i=0}^{n-1} a_i x^i$, or in the form of $\mathbf{a} = (a_0, a_1, \ldots, a_{n-1})$ vector space over Zq, where $a_i \in \mathbb{Z}_q$.

Main target rings are $\mathbb{Z}_q[x]/(x^n - 1)$ and $\mathbb{Z}_q[x]/(x^n + 1)$, since they are widely used in lattice-based schemes.

Let n be a power of two, then $x^n + 1$ is the $2n$-th cyclotomic polynomial.

- $n = 2^k$일 때, $\phi_{2n}(x) = x^n + 1$ 증명

  1. 사전 증명

$$x^n - 1 = \prod_{d|n} \Phi_d(x)$$

이므로, 소수 p와 자연수 $r \geq 1$에 대해 $n = p^r$일 때, 약수들은 $1, p, p^2, \cdots, p^r$이고

$$x^{p^r} - 1 = \prod_{j=0}^{r} \Phi_{p^j}(x) = \Phi_{p^r}(x) \cdot \prod_{j=0}^{r-1} \Phi_{p^j}(x) = \Phi_{p^r}(x)(x^{p^{r-1}} - 1)$$

이므로

$$\Phi_{p^r}(x) = \frac{x^{p^r} - 1}{x^{p^{r-1}} - 1}$$

이다.

한편,

$$\Phi_p(y) = \frac{y^p - 1}{y - 1}$$

이므로 $y = x^{p^{r-1}}$를 대입하면

$$\Phi_p(x^{p^{r-1}}) = \frac{(x^{p^{r-1}})^p - 1}{x^{p^{r-1}} - 1} = \frac{x^{p^r} - 1}{x^{p^{r-1}} - 1}$$

이다. 따라서

$$\Phi_{p^r}(x) = \Phi_p(x^{p^{r-1}})$$

이다. 즉, p^r-th cyclotomic polynomial은 p-th cyclotomic polynomial의 변수에 $x^{p^{r-1}}$을 대입한 것과 같다.

  2. 본 증명

  $n = 2^k$이므로 $(k \geq 0)$

$$\Phi_{2n}(x) = \Phi_{2^{k+1}}(x) = \Phi_2(x^{2^{(k+1)-1}}) = \Phi_2(x^{2^k})$$

  이다.

  2-th cyclotomic polynomial은

$$\Phi_2(y) = \frac{y^2 - 1}{y - 1} = y + 1$$

  이므로, y에 $x^{2^k} = x^n$를 대입하면

$$\Phi_{2n}(x) = \Phi_2(x^{2^k}) = \Phi_2(x^n) = x^n + 1$$

  이다.

  결과적으로 $2n = 2^{k+1}$-th cyclotomic polynomial은 $x^n + 1$이다.

1.3 [+] CRT for the Polynomial Ring

Theorem

$f_1(x), f_2(x), \ldots, f_k(x)$가 $F[x]$의 원소이고, 모든 $i \neq j$에 대해 $\gcd(f_i(x), f_j(x)) = 1$이라고 가정하자.

$f(x) = f_1(x)f_2(x)\cdots f_k(x)$ 라고 할 때, 다음 ring Isomorphism이 성립한다.

$$F[x]/\langle f(x)\rangle \;\cong\; F[x]/\langle f_1(x)\rangle \times F[x]/\langle f_2(x)\rangle \times \cdots \times F[x]/\langle f_k(x)\rangle$$

두 ring $(R, +, \cdot)$과 $(S, \oplus, \odot)$ 사이에 함수 $\phi : R \to S$가 존재하고, 다음 세 조건을 만족하면 ϕ를 Ring Isomorphism이라고 한다.

1. 전단사(Bijective)

   ϕ는 일대일 대응(One-to-one correspondence)이어야 한다.

   즉, 단사(Injective)이면서 전사(Surjective)여야 한다. (1-1, onto)

2. 덧셈 보존 (Homomorphism - Additive)

   임의의 $a, b \in R$에 대해, $\phi(a + b) = \phi(a) \oplus \phi(b)$

3. 곱셈 보존 (Homomorphism - Multiplicative)

   임의의 $a, b \in R$에 대해, $\phi(a \cdot b) = \phi(a) \odot \phi(b)$

이는 임의의 다항식 $a_1(x), \ldots, a_k(x)$가 주어졌을 때, 다음 연립 합동식을 만족하는 다항식 $A(x)$가 modulus $f(x)$에 대해 유일하게 존재함을 의미한다.

$$\begin{aligned} A(x) &\equiv a_1(x) \pmod{f_1(x)} \\ A(x) &\equiv a_2(x) \pmod{f_2(x)} \\ &\vdots \\ A(x) &\equiv a_k(x) \pmod{f_k(x)} \end{aligned}$$

Proof

1. map 정의

2. Homomorphism

3. Injective

4. Surjective

1. Define the Map

   함수 $\phi : F[x] \to F[x]/\langle f_1(x)\rangle \times \cdots \times F[x]/\langle f_k(x)\rangle$ 를 다음과 같이 정의한다.

$$\phi(h(x)) = (h(x) \bmod f_1(x),\ \ldots,\ h(x) \bmod f_k(x))$$

2. Homomorphsim

이 함수 ϕ는 다항식의 덧셈과 곱셈을 보존하므로 Ring Homomorphism(덧셈 보존, 곱셈 보존, indentity 보존)이다.

3. Injective

   이 함수가 단사임을 보이려면 "함수값 $\phi(h(x))$가 $(0,\ldots,0)$ 이라면, $h(x)$가 반드시 0이어야 함"을 보이면 된다.

   (a) Kernel이 공집합이 아니라고 가정하자.

   즉, 어떤 다항식 $h(x)$가 quatient ring $F[x]/\langle f(x)\rangle$의 원소이고,

   $$\phi(h(x)) = (0,0,\ldots,0)$$

   라고 하자.

   (b) 함수의 정의에 따라, 이는 $h(x)$를 각 $f_i(x)$로 나누었을 때 나머지가 모두 0이라는 뜻이다.

   즉,

   $$h(x) \equiv 0 \pmod{f_1(x)}$$

   $$h(x) \equiv 0 \pmod{f_2(x)}$$

   $$\vdots$$

   $$h(x) \equiv 0 \pmod{f_k(x)}$$

   (c) 모든 $f_i(x)$는 서로소이므로, 개별적인 $f_i(x)$로 나누어 떨어지면, 그들의 곱인 $f(x)$로도 나누어 떨어진다.

   (d) $h(x)$가 $f(x)$의 배수라는 말은, $F[x]/\langle f(x)\rangle$에서 0과 동치라는 뜻이다.

   $$f(x) = f_1(x)f_2(x)\cdots f_k(x) \mid h(x)$$

   $$\implies h(x) \equiv 0 \pmod{f(x)}$$

   이는 $\phi(h(x)) = (0,..,0)$이 되는 $h(x)$가 0밖에 없다는 것이므로

   $$\mathrm{Ker}(\phi) = \{0\}$$

   이고, Kernel이 0 하나뿐이므로, 이 함수는 단사(Injective)이다.

- Kernel 을 통한 동형 사상 구하기

  ϕ의 kernel이 $\langle f(x)\rangle$이면 $F[x]/\langle f(x)\rangle \cong \mathrm{Im}(\phi)$이므로 ϕ의 Kernel을 구해 이를 인한다.

  $\phi(h(x)) = (0,\ldots,0)$ 이 되려면 모든 i에 대해 다음이 성립해야 한다.

  $$h(x) \equiv 0 \pmod{f_i(x)}$$

  즉, $h(x)$는 모든 $f_i(x)$로 나누어 떨어져야 한다.

  $f_i(x)$들이 쌍마다 서로 소(pairwise coprime)이므로, $h(x)$는 그들의 곱인 $f(x)$로 나누어 떨어져야 한다.

  $$\mathrm{Ker}(\phi) = \{h(x) \in F[x] \mid f(x) \text{ divides } h(x)\} = \langle f(x)\rangle$$

  따라서, First Isomorphism Theorem에 의해 다음이 성립한다.

  $$F[x]/\langle f(x)\rangle = F[x]/\mathrm{Ker}(\phi) \cong \mathrm{Im}(\phi)$$

4. Surjectivity

   오른쪽 ring의 임의의 원소 $(a_1(x),\ldots,a_k(x))$에 대해, $\phi(A(x)) = (a_1(x),\ldots,a_k(x))$를 만족하는 $A(x)$가 존재함을 보여야 한다.

각 i $(1 \leq i \leq k)$에 대해 다음을 정의한다.

$$M_i(x) = \frac{f(x)}{f_i(x)} = \prod_{j \neq i} f_j(x)$$

$f_j(x)$들이 서로 소이므로, $M_i(x)$와 $f_i(x)$는 서로 소이다($\gcd(M_i(x), f_i(x)) = 1$).

따라서 Extended Euclidean Algorithm에 의해 다음을 만족하는 다항식 $s_i(x), t_i(x)$가 존재한다.

$$s_i(x)M_i(x) + t_i(x)f_i(x) = 1$$

위 식을 modulus법 $f_i(x)$로 보면

$$s_i(x)M_i(x) \equiv 1 \pmod{f_i(x)}$$

이고, $e_i(x) = s_i(x)M_i(x)$ 라고 정의하면, $e_i(x)$는 다음과 같은 성질을 가진다.

- $e_i(x) \equiv 1 \pmod{f_i(x)}$
- $e_i(x) \equiv 0 \pmod{f_j(x)}$ (단, $j \neq i$)
  ($M_i(x)$가 $f_j(x)$를 인수로 포함하므로)

이제 해 $A(x)$를 다음과 같이 구성한다.

$$A(x) = \sum_{i=1}^{k} a_i(x)e_i(x) = \sum_{i=1}^{k} a_i(x)s_i(x)M_i(x)$$

이 $A(x)$를 $f_j(x)$로 나눈 나머지를 확인하면

$$A(x) \equiv a_j(x) \cdot 1 + \sum_{i \neq j} a_i(x) \cdot 0 \equiv a_j(x) \pmod{f_j(x)}$$

이다. 따라서 $A(x)$는 주어진 모든 합동식을 만족한다.

$\therefore$ ϕ를 통해 유도된 사상 $\bar{\phi} : F[x]/\langle f(x)\rangle \to \prod F[x]/\langle f_i(x)\rangle$ 는 단사(Injective)이며 전사(Surjective)이므로 동형(Isomorphism)이다.

- Uniqueness (유일성 증명)

  덧붙여, 만약 두 다항식 $A(x)$와 $B(x)$가 모두 주어진 연립 합동식을 만족한다고 가정하자. 즉, 모든 $i = 1, \ldots, k$에 대하여 다음이 성립한다.

$$A(x) \equiv a_i(x) \pmod{f_i(x)}$$

$$B(x) \equiv a_i(x) \pmod{f_i(x)}$$

두 식을 빼면,

$$A(x) - B(x) \equiv 0 \pmod{f_i(x)}$$

즉, $D(x) = A(x) - B(x)$는 모든 $f_i(x)$로 나누어 떨어진다.

$f_i(x)$들은 서로소(pairwise coprime)이므로, $D(x)$는 그들의 곱인 $f(x) = f_1(x) \cdots f_k(x)$로 나누어 떨어져야 한다.

$$A(x) - B(x) \equiv 0 \pmod{f(x)}$$

$$\therefore A(x) \equiv B(x) \pmod{f(x)}$$

따라서 modulus $f(x)$에 대한 해 $A(x)$는 유일하다.

Example

다음 연립 합동식을 만족하는 다항식 $A(x)$를 구하시오.

$$\begin{aligned} &1. \quad A(x) \equiv 2 \pmod{x-1} \\ &2. \quad A(x) \equiv 2x+1 \pmod{x^2+1} \end{aligned}$$

- $f_1(x) = x-1,\ f_2(x) = x^2+1$

  $\Rightarrow \gcd(x-1, x^2+1) = 1$이므로 CRT 적용 가능

- $f(x) = (x-1)(x^2+1) = x^3 - x^2 + x - 1$
- $a_1(x) = 2,\ a_2(x) = 2x+1$

$M_1(x) = f(x)/f_1(x) = x^2+1$

$M_2(x) = f(x)/f_2(x) = x-1$

1. $M_1^{-1}(x) \pmod{x-1}$

   $M_1(x) = x^2+1$

   $M_1(1) = 1^2+1 = 2$

   $\rightarrow 2 \cdot s_1 \equiv 1 \pmod{x-1}$이 되는 상수 s_1을 찾아야 함

   $\mathbb{Q}$에서 2의 역원은 $1/2$

   $\therefore M_1^{-1}(x) = 1/2$

2. $M_2^{-1}(x) \pmod{x^2+1}$

   $M_2(x) = x-1$

   x^2+1이므로

   x^2+1을 $x-1$로 나누면

   $$x^2+1 = (x+1)(x-1)+2$$

   로 나머지가 2이며, 식을 정리하면

   $$2 = (x^2+1) - (x+1)(x-1)$$

   $$1 = \frac{1}{2}(x^2+1) - \frac{1}{2}(x+1)(x-1)$$

   $$1 \equiv -\frac{1}{2}(x+1) \cdot (x-1) \pmod{x^2+1}$$

   따라서 $(x-1)$의 역원은 $-1/2(x+1)$

   $\therefore M_2^{-1}(x) = -1/2(x+1)$

3. 해 $A(x)$ 구성

$$A(x) = a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1}$$

$$
\begin{aligned}
A(x) &= 2 \cdot (x^2+1) \cdot \left(\frac{1}{2}\right) + (2x+1) \cdot (x-1) \cdot \left[-\frac{1}{2}(x+1)\right] \\
&= (x^2+1) - \frac{1}{2}(2x+1)(x^2-1) \quad (\because (x-1)(x+1) = x^2-1) \\
&= (x^2+1) - \frac{1}{2}(2x^3 - 2x + x^2 - 1) \\
&= x^2 + 1 - x^3 + x - \frac{1}{2}x^2 + \frac{1}{2} \\
&= -x^3 + \frac{1}{2}x^2 + x + \frac{3}{2}
\end{aligned}
$$

4. Modulus $f(x)$로 reduction

   구한 해의 차수가 $f(x)$ 3차보다 크거나 같으므로, $f(x) = x^3 - x^2 + x - 1$로 나누어 나머지를 구해야 한다.

   $A(x)$ 를 $x^3 - x^2 + x - 1$ 로 나눈 나머지를 구한다.

   $x^3 \equiv x^2 - x + 1 \pmod{f(x)}$ 관계를 이용해 대입한다.

$$
\begin{aligned}
A(x) &\equiv -(x^2 - x + 1) + \frac{1}{2}x^2 + x + \frac{3}{2} \\
&\equiv -x^2 + x - 1 + \frac{1}{2}x^2 + x + \frac{3}{2} \\
&\equiv -\frac{1}{2}x^2 + 2x + \frac{1}{2}
\end{aligned}
$$

Verification

구한 해 $A(x) = -1/2x^2 + 2x + 1/2$ 가 조건을 만족하는지 확인한다.

1. 첫 번째 조건 $(x-1)$: $x = 1$ 대입

$$
\begin{aligned}
A(1) &= -\frac{1}{2} + 2 + \frac{1}{2} = 2 \\
&\to A(x) \equiv 2 \pmod{x-1}
\end{aligned}
$$

2. 두 번째 조건 (x^2+1): $x^2 \equiv -1$ 대입

$$
\begin{aligned}
A(x) &\equiv -\frac{1}{2}(-1) + 2x + \frac{1}{2} = \frac{1}{2} + 2x + \frac{1}{2} = 2x + 1 \\
&\to A(x) \equiv 2x + 1 \pmod{x^2+1}
\end{aligned}
$$

1.4 Polynomial multiplication and Convolution

Without loss of generality, $\boldsymbol{a}$ and $\boldsymbol{b}$ of degree $n-1$ are considered in this paper. Pad them with zero if their lengths are less than n.

- **Linear** convolution

  Consider $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]$, then

$$
\boldsymbol{c} = \sum_{k=0}^{2n-2} c_k x^k \in \mathbb{Z}_q[x],
$$

$$
\text{where } c_k = \sum_{i+j=k} a_i b_j \bmod q \quad (k = 0, 1, \ldots, 2n-2).
$$

Here, c is referred to as the linear convolution of $\boldsymbol{a}$ and $\boldsymbol{b}$.

Consider $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/\langle \phi(x) \rangle$. One can first compute $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]$, then $\boldsymbol{c} = \boldsymbol{c}' \bmod \phi(x)$.

- **Cyclic** convolution

  Consider $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n - 1)$, then

$$\boldsymbol{c} = \sum_{k=0}^{n-1} c_k x^k,$$

$$\text{where } c_k = \sum_{i=0}^{k} a_i b_{k-i} + \sum_{i=k+1}^{n-1} a_i b_{n+k-i} \bmod q \quad (k = 0, 1, \ldots, n-1).$$

  And $\boldsymbol{c}$ is referred to as the cyclic convolution (CC for short) of $\boldsymbol{a}$ and $\boldsymbol{b}$.

- **Negative wrapped** convolution

  Consider $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n + 1)$, then

$$\boldsymbol{c} = \sum_{k=0}^{n-1} c_k x^k,$$

$$\text{where } c_k = \sum_{i=0}^{k} a_i b_{k-i} - \sum_{i=k+1}^{n-1} a_i b_{n+k-i} \bmod q \quad (k = 0, 1, \ldots, n-1).$$

  Here, $\boldsymbol{c}$ is referred to as their negative wrapped convolution (NWC for short).

  ※ $x^{n+k} = x^n \cdot x^k \equiv (-\mathbf{1}) \cdot x^k = -x^k$

2 NUMBER THEORETIC TRANSFORM (NTT)

NTT is the special case of the discrete Fourier transform (DFT) over a finite field.

NTT and DFT share the same formula and similar properties, except that DFT has complex twiddle factors $\exp(-2\pi i/n)$, while NTT uses an integer primitive root of unity ω_n.

2.1 Cyclic Convolution-based NTT

n-point CC-based NTT has two parameters.

- The length or the point n, where n is a power of two.
- The modulus q, where q is a prime number satisfying $q \equiv 1 \pmod{n}$.
- It implies that the primitive n-th root of unity ω_n in $\mathbb{Z}_q$ exists.

> $\mathbb{Z}_q$가 소수 q에 대한 Field이므로,
> 0을 제외한 곱셈군 $\mathbb{Z}_q^* = 1, 2, \ldots, q-1$은 order $= q-1$인 group이다.
> $\mathbb{Z}_q$의 곱셈군 $\mathbb{Z}_q^*$는 항상 cyclic group이므로,
> 어떤 generator g가 존재하여 $\mathbb{Z}_q^* = \{g^1, g^2, \ldots, g^{q-1}\}$이다.
> 이때 $q \equiv 1 \pmod{n}$이므로 $n \mid (q-1)$이고,
> 따라서 $\mathbb{Z}_q$에는 order $= n$인 cyclic subgroup의 generator가 존재한다.
> order $= n$인 $x \in \mathbb{Z}_q$는 다음을 만족한다.
> - $x^n \equiv 1 \pmod{q}$

- $0 < k < n$ 인 모든 k에 대해 $x^k \not\equiv 1 \pmod{q}$
즉, x는 primitive n-th root of unity이다.
따라서 소수 q에 대해 $q \equiv 1 \pmod{n}$이라면, $\mathbb{Z}_q$에는 primitive n-th root of unity ω_n가 존재하며,
특히, $\phi(n)$개만큼 존재한다.

The forward transform, denoted by NTT, is defined as: $\hat{\boldsymbol{a}} = \mathrm{NTT}(\boldsymbol{a})$, where

$$\hat{a}_j = \sum_{i=0}^{n-1} a_i \omega_n^{ij} \bmod q \quad (j = 0, 1, \ldots, n-1).$$

NTT의 matrix 표현

$$NTT = \begin{bmatrix} \omega^{0\times 0} & \cdots & \omega^{0\times j} & \cdots & \omega^{0\times(n-1)} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \omega^{i\times 0} & \cdots & \omega^{i\times j} & \cdots & \omega^{i\times(n-1)} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \omega^{(n-1)\times 0} & \cdots & \omega^{(n-1)\times j} & \cdots & \omega^{(n-1)\times(n-1)} \end{bmatrix}$$

The inverse transform, denoted by INTT, is defined as $\boldsymbol{a} = \mathrm{INTT}(\hat{\boldsymbol{a}})$, where

$$a_i = n^{-1} \sum_{j=0}^{n-1} \hat{a}_j \omega_n^{-ij} \bmod q \quad (i = 0, 1, \ldots, n-1).$$

INTT의 matrix 표현

$$NTT^{-1} = \frac{1}{n} \begin{bmatrix} \omega^{-[0\times 0]} & \cdots & \omega^{-[0\times j]} & \cdots & \omega^{-[0\times(n-1)]} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \omega^{-[i\times 0]} & \cdots & \omega^{-[i\times j]} & \cdots & \omega^{-[i\times(n-1)]} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \omega^{-[(n-1)\times 0]} & \cdots & \omega^{-[(n-1)\times j]} & \cdots & \omega^{-[(n-1)\times(n-1)]} \end{bmatrix}$$

$$= \frac{1}{n} \begin{bmatrix} \omega^{[0\times 0]+(n/2)} & \cdots & \omega^{[0\times j]+(n/2)} & \cdots & \omega^{[0\times(n-1)]+(n/2)} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \omega^{[i\times 0]+(n/2)} & \cdots & \omega^{[i\times j]+(n/2)} & \cdots & \omega^{[i\times(n-1)]+(n/2)} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \omega^{[(n-1)\times 0]+(n/2)} & \cdots & \omega^{[(n-1)\times j]+(n/2)} & \cdots & \omega^{[(n-1)\times(n-1)]+(n/2)} \end{bmatrix}$$

Note that the inverse transform can be implemented by replacing the ω_n in NTT procedure with ω_n^{-1}, followed by multiplying by a scale factor n^{-1}.

Prop)

It always holds that $\boldsymbol{a} = \mathrm{INTT}(\mathrm{NTT}(\boldsymbol{a}))$.

Prop) Cyclic Convolution Property

Let $\boldsymbol{c}$ be the cyclic convolution of $\boldsymbol{a}$ and $\boldsymbol{b}$, then it holds that

$$\mathrm{NTT}(\boldsymbol{c}) = \mathrm{NTT}(\boldsymbol{a} \cdot \boldsymbol{b}) = \mathrm{NTT}(\boldsymbol{a}) \circ \mathrm{NTT}(\boldsymbol{b}).$$

2.2 Negative Wrapped Convolution-based NTT

The modulus q is set to be a prime number satisfying $q \equiv 1 \pmod{2n}$ such that the primitive $2n$-th root of unity ψ_{2n} in $\mathbb{Z}_q$ exists.

[왜 $2n$-th root of unity를 구하는가?]
Negative Wrapped Convolution은 $\mathbb{Z}_q[x]/(x^n+1)$에서 연산을 수행하므로 $x^n \equiv -1$가 되는 x를 구해야 한다.
(n-th primitive roots of unity는 $x^n = 1$이므로 구하고자 하는 근이 아님)
이때 x는 $x^{2n} \equiv 1$이므로 $q \equiv 1 \pmod{2n}$로 설정하면, $(\psi_{2n}^n)^2 \equiv 1 \implies \psi_{2n}^n \equiv -1$인 primitive $2n$-th root of unity $\psi_{2n} \in \mathbb{Z}_q$를 구할 수 있다.
이때, ψ_{2n}는 primitive이므로 $k < 2n$에 대하여 $\psi_{2n}^k \neq 1$이므로 $\psi_{2n}^n \equiv 1$이다.
※ $\mathbb{Z}_q[x]/(x^{2n}-1)$의 primitive $2n$-th roots of unity ψ_{2n}

중국인의 나머지 정리에 의해

$$\mathbb{Z}_q[x]/(x^{2n}-1) \cong \underbrace{\mathbb{Z}_q[x]/(x^n-1)}_{\text{Cyclic Convolution}} \times \underbrace{\mathbb{Z}_q[x]/(x^n+1)}_{\text{Negative Wrapped}}$$

이고, $\mathbb{Z}_q[x]/(x^{2n}-1)$에서 구한 $2n$개 근은 $\{\psi_{2n}^0, \psi_{2n}^1, \psi_{2n}^2, \ldots, \psi_{2n}^{2n-1}\}$이다.
이때, $k = 0, 1, \ldots, n-1$에 대하여 ψ_{2n}^{2k}는

$$(\psi_{2n}^{2k})^n = (\psi_{2n}^{2n})^k = 1^k = 1$$

이므로 $\mathbb{Z}_q[x]/(x^n-1)$의 근이 되고, ψ_{2n}^{2k+1}은

$$(\psi_{2n}^{2k+1})^n = (\psi_{2n}^{2n})^k \cdot \psi_{2n}^n = 1 \cdot (-1) = -1$$

이므로 $\mathbb{Z}_q[x]/(x^n+1)$의 근이 된다.

Take $\omega_n = \psi_{2n}^2 \bmod q$, and write

$$\boldsymbol{\psi} = (1, \psi_{2n}, \psi_{2n}^2, \ldots, \psi_{2n}^{n-1}), \quad \boldsymbol{\psi}^{-1} = (1, \psi_{2n}^{-1}, \psi_{2n}^{-2}, \ldots, \psi_{2n}^{-(n-1)}).$$

Define

$$\bar{\boldsymbol{a}} = \boldsymbol{\psi} \circ \boldsymbol{a}, \text{ where } \bar{a}_i = \psi_{2n}^i a_i$$

in detail, which implies

$$\boldsymbol{a} = \boldsymbol{\psi}^{-1} \circ \bar{\boldsymbol{a}}, \text{ where } a_i = \psi_{2n}^{-i} \bar{a}_i.$$

n-point NWC-based NTT is to integrate $\boldsymbol{\psi}$ (resp., $\boldsymbol{\psi}^{-1}$) into NTT (resp., INTT), and denote them by NTT^{ψ} (resp., $\mathrm{INTT}^{\psi^{-1}}$), that is

$$\hat{\boldsymbol{a}} = \mathrm{NTT}^{\psi}(\boldsymbol{a}) = \mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{a}),$$

$$\boldsymbol{a} = \mathrm{INTT}^{\psi^{-1}}(\hat{\boldsymbol{a}}) = \boldsymbol{\psi}^{-1} \circ \mathrm{INTT}(\hat{\boldsymbol{a}}).$$

matrix 표현은 CC와 동일 (psi가 아닌 omega 사용)

More specifically, the forward transform $\hat{\boldsymbol{a}} = \mathrm{NTT}^{\psi}(\boldsymbol{a})$ can be written as

$$\hat{a}_j = \sum_{i=0}^{n-1} a_i \psi_{2n}^{i} \omega_n^{ij} \bmod q \quad (j = 0, 1, \ldots, n-1).$$

The inverse transform $\boldsymbol{a} = \mathrm{INTT}^{\psi^{-1}}(\hat{\boldsymbol{a}})$ can be written as

$$a_i = n^{-1} \psi_{2n}^{-i} \sum_{j=0}^{n-1} \hat{a}_j \omega_n^{-ij} \bmod q \quad (i = 0, 1, \ldots, n-1).$$

Prop)

It always holds that $\boldsymbol{a} = \mathrm{INTT}^{\psi^{-1}}(\mathrm{NTT}^{\psi}(\boldsymbol{a}))$.

Prop) Negative Wrapped Convolution Property

Let $\boldsymbol{c}$ be the negative wrapped convolution of $\boldsymbol{a}$ and $\boldsymbol{b}$, then it holds that

$$\begin{aligned}
\mathrm{NTT}^{\psi}(\boldsymbol{c}) &= \mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{c}) \\
&= \mathrm{NTT}(\boldsymbol{\psi} \circ (\underbrace{\boldsymbol{a} \cdot \boldsymbol{b}}_{\mathrm{NWC}})) \\
&= \mathrm{NTT}(\underbrace{(\boldsymbol{\psi} \circ \boldsymbol{a}) \cdot (\boldsymbol{\psi} \circ \boldsymbol{b})}_{\mathrm{CC}\ \because\ \psi_{wn}^{2k+1} \to\ \psi_{wn}^{2k+2}}) \\
&= \mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{a}) \circ \mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{b}) \\
&= \mathrm{NTT}^{\psi}(\boldsymbol{a}) \circ \mathrm{NTT}^{\psi}(\boldsymbol{b}).
\end{aligned}$$

Example

Initialization

- $\mathbb{Z}_q[x]/(x^n + 1)$

  - $n = 2$, modulus $x^2 + 1$ (즉, $x^2 \equiv -1$)
  - $q = 5$ ($q \equiv 1 \pmod{2n}$ 만족)

- $\psi = 2$

  $2^1 = 2,\ 2^2 = 4,\ 2^3 = 8 \equiv 3,\ 2^4 = 16 \equiv 1,\ \psi^n \equiv -1 \pmod{q}$

- $\omega = 4$
- NTT matrix $N = \begin{bmatrix} 1 & 1 \\ 1 & \omega \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix}$
- $\boldsymbol{a} = [1, 1]$ (즉, $x + 1$)
- $\boldsymbol{b} = [2, 1]$ (즉, $x + 2$)

Calculation (벡터표현과 맞추기 위해 다항식을 저차항부터 표기)

$\boldsymbol{\psi} = [\psi^0, \psi^1] = [1, 2]$

1. $\mathrm{NTT}^{\psi}(\boldsymbol{c}) := \mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{c}) = \mathrm{NTT}(\boldsymbol{\psi} \circ (\boldsymbol{a} \cdot \boldsymbol{b}))$

   $\boldsymbol{c} = (1 + x)(2 + x) = 2 + 3x + x^2 \equiv 2 + 3x + (-1) = 1 + 3x \implies [1, 3]$

   $\boldsymbol{\psi} \circ \boldsymbol{c} = [1, 2] \circ [1, 3] = [1, 6] \equiv [1, 1]$

$$\mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{c}) = N([1, 1]) = \begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix} \equiv \begin{bmatrix} 2 \\ 0 \end{bmatrix} = [2, 0]$$

   $\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{c} = [1, 3]$

   $\mathrm{NTT}(\boldsymbol{\psi} \circ (\boldsymbol{a} \cdot \boldsymbol{b})) = [2, 0]$

2. $\mathrm{NTT}((\boldsymbol{\psi} \circ \boldsymbol{a}) \cdot (\boldsymbol{\psi} \circ \boldsymbol{b}))$

   ※ $\boldsymbol{\psi}$를 곱해줬기 때문에 Cyclic convolution 공간이 됨 ⇒ $x^2 \equiv 1$

   $\boldsymbol{\psi} \circ \boldsymbol{a} = [1, 2] \circ [1, 1] = [1, 2] \implies 1 + 2x$

   $\boldsymbol{\psi} \circ \boldsymbol{b} = [1, 2] \circ [2, 1] = [2, 2] \implies 2 + 2x$

$$\begin{aligned}(\boldsymbol{\psi} \circ \boldsymbol{a}) \cdot (\boldsymbol{\psi} \circ \boldsymbol{b}) &= (1 + 2x)(2 + 2x) = 2 + 4x + 2x + 4x^2 \\ &= 2 + 6x + 4(1) = 6 + 6x \\ &\equiv 1 + x \implies [1, 1]\end{aligned}$$

$$\mathrm{NTT}((\boldsymbol{\psi} \circ \boldsymbol{a}) \cdot (\boldsymbol{\psi} \circ \boldsymbol{b})) = N([1, 1]) = \begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix} \equiv \begin{bmatrix} 2 \\ 0 \end{bmatrix} = [2, 0]$$

3. $\mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{a}) \circ \mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{b}) := \mathrm{NTT}^{\psi}(\boldsymbol{a}) \circ \mathrm{NTT}^{\psi}(\boldsymbol{b})$

$$\mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{a}) = N([1, 2]) = \begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 9 \end{bmatrix} \equiv \begin{bmatrix} 3 \\ 4 \end{bmatrix} = [3, 4]$$

$$\mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{b}) = N([2, 2]) = \begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 10 \end{bmatrix} \equiv \begin{bmatrix} 4 \\ 0 \end{bmatrix} = [4, 0]$$

$$\mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{a}) \circ \mathrm{NTT}(\boldsymbol{\psi} \circ \boldsymbol{b}) = [3, 4] \circ [4, 0] = [12, 0] = [2, 0]$$

2.3 NTT-based Polynomial Multiplication

NTT can be used to compute inear/cyclic/negative-wrapped convolutioLns.

2.3.1 Linear convolution-based polynomial multiplication

To compute the linear convolution $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}q[x]$,

first, pad them to the length of $2n$ with zeros, resulting with

$$\boldsymbol{a}' = (a_0, \ldots, a_{n-1}, \underbrace{0, \ldots, 0}_{\#\,0\,=\,n}) \text{ and } \boldsymbol{b}' = (b_0, \ldots, b_{n-1}, \underbrace{0, \ldots, 0}_{\#\,0\,=\,n}).$$

Second, use $2n$-point NTT/INTT for

$$\boldsymbol{c} = \text{INTT}(\text{NTT}(\boldsymbol{a}') \circ \text{NTT}(\boldsymbol{b}')).$$

Moreover, to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(\phi(x))$, one can compute $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]$ with $2n$-point NTT/INTT, followed by computing

$$\boldsymbol{c} = \boldsymbol{c}' \bmod \phi(x).$$

2.3.2 Cyclic convolution-based polynomial multiplication

To compute the cyclic convolution $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n - 1)$, one can straightly use n-point NTT/INTT, according to cyclic convolution property

$$\boldsymbol{c} = \text{INTT}(\text{NTT}(\boldsymbol{a}) \circ \text{NTT}(\boldsymbol{b})).$$

2.3.3 Negative wrapped convolution-based polynomial multiplication

To compute the negative wrapped convolution $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n + 1)$, one can use the negative wrapped convolution property

$$\boldsymbol{c} = \text{INTT}^{\psi^{-1}}\left(\text{NTT}^{\psi}(\boldsymbol{a}) \circ \text{NTT}^{\psi}(\boldsymbol{b})\right).$$

2.3.4 Complexity

The complexity of directly computing NTT/INTT is $O(n^2)$.

There are

- two NTTs,
- one point-wise multiplication and
- one INTT for NTT-based multiplication.

Therefore, the complexity of NTT-based multiplication without fast algorithms is $O(n^2)$.

2.3.5 Advantages of NTT

Here, let NTT/INTT be any kind of forward/inverse transforms.

- Firstly, both NTT and INTT are linear transformations, based on which it can save INTTs in lattice-based schemes (e.g., Kyber and Dilithium), i.e.,

$$\begin{aligned}\sum_{i=0}^{l} \boldsymbol{a}_i \boldsymbol{b}_i &= \sum_{i=0}^{l} \mathrm{INTT}(\mathrm{NTT}(\boldsymbol{a}_i) \circ \mathrm{NTT}(\boldsymbol{b}_i)) \\ &= \mathrm{INTT}(\sum_{i=0}^{l} \mathrm{NTT}(\boldsymbol{a}_i) \circ \mathrm{NTT}(\boldsymbol{b}_i)).\end{aligned}$$

- Additionally, consider $\boldsymbol{c} = \mathrm{INTT}(\mathrm{NTT}(\boldsymbol{a}) \circ \mathrm{NTT}(\boldsymbol{b}))$, where $\boldsymbol{a}$ is random. Since the NTT transforms keep the randomness of a random polynomial, i.e., $\hat{\boldsymbol{a}} = \mathrm{NTT}(\boldsymbol{a})$ is also random, one can directly generate a random polynomial, and view it as random $\hat{\boldsymbol{a}}$ already in the NTT domain, and compute $\boldsymbol{c} = \mathrm{INTT}(\hat{\boldsymbol{a}} \circ \mathrm{NTT}(\boldsymbol{b}))$, thus eliminating the need for the forward transform.

- Besides, NTT and INTT preserve the dimension and bit length of all individual coefficients of a polynomial, i.e., $\hat{\boldsymbol{a}}$ and $\boldsymbol{a}$ share the same dimension and bit length of any coefficient. Thus, $\hat{\boldsymbol{a}}$ can be stored where $\boldsymbol{a}$ is originally placed.

- Finally, in some cases where $\boldsymbol{a}$ is involved in multiple multiplications, $\hat{\boldsymbol{a}}$ is computed once and stored for its use in subsequent multiplications, which can save forward transforms without any extra requirement of storage.

2.4 SOME TRICKS FOR POLYNOMIAL MULTIPLICATION

Def) One-iteration Karatsuba algorithm

Let a, b, c, d be any numbers or polynomials.

The one-iteration Karatsuba algorithm

1 word를 X라고 할 때

$$\begin{aligned}f &= aX + b, \quad g = cX + d \\ fg &= (aX + b)(cX + d) \\ &= acX^2 + (ad + bc)X + bd \\ &= t_1 X^2 + t_2 X + t_3\end{aligned}$$

implies that, to compute

$$t_1 = a \cdot c,\ t_2 = a \cdot d + b \cdot c \quad \text{and} \quad t_3 = b \cdot d,$$

first compute t_1 and t_3, and then compute t_2 by

$$t_2 = (a + b) \cdot (c + d) - t_1 - t_3.$$

One-iteration Karatsuba algorithm saves one multiplication at the cost of three extra additions (subtractions).

$$
\begin{aligned}
t_2 &= ad + bc && \text{2 multiplications} \\
&= ad + bc + ac - ac + bd - bd \\
&= ac + bc + ad + bd - ac - bd && \downarrow \\
&= (a+b)(c+d) - ac - bd \\
&= (a+b)(c+d) - t_1 - t_2 && \text{1 multiplication}
\end{aligned}
$$

Def) Good's trick

As for the ring $\mathbb{Z}_q[x]/(x^{h\cdot 2^k} - 1)$ where h is an odd number, Good's trick maps

$$\mathbb{Z}_q[x]/(x^{h\cdot 2^k} - 1) \xrightarrow{\phi} \left(\mathbb{Z}_q[z]/(z^{2^k} - 1)\right)[y]/(y^h - 1),$$

$$\text{where } \boldsymbol{a} = \sum_{l=0}^{h\cdot 2^k - 1} a_l x^l \mapsto \sum_{l=0}^{h\cdot 2^k - 1} a_l y^{(l \bmod h)} z^{(l \bmod 2^k)}$$
$$= \sum_{i=0}^{h-1} \sum_{j=0}^{2^k-1} \tilde{a}_{i,j} y^i z^j.$$

원리: 중국인의 나머지 정리

Good's trick의 가장 중요한 조건은 $N = h \cdot 2^k$에서 h가 홀수라는 점으로, $\gcd(h, 2^k) = 1$이다.

CRT에 의해, 서로소인 두 정수 $h, 2^k$에 대해 다음 Group Isomorphism이 성립한다.

$$\mathbb{Z}_{h\cdot 2^k} \cong \mathbb{Z}_h \times \mathbb{Z}_{2^k}$$

이 동형 사상을 통해 l을 다음과 같이 매핑한다.

$$l \mapsto (l \bmod h,\ l \bmod 2^k) = (i, j)$$

$$l = \underbrace{i \cdot [2^k \cdot ((2^k)^{-1} \bmod h)]}_{i \text{ component}} + \underbrace{j \cdot [h \cdot (h^{-1} \bmod 2^k)]}_{j \text{ component}} \pmod{h \cdot 2^k}$$

$i = l \bmod h$를 차수로 갖는 변수(formal variable)를 y, $j = l \bmod 2^k$를 차수로 갖는 변수를 z라고 하면, x라는 하나의 변수가 y, z라는 두 개의 변수로 분리되면서, 1차원 다항식이 행렬 형태의 2차원 다항식으로 변환된다.

$$\phi(x^l) = y^{(l \bmod h)} \cdot z^{(l \bmod 2^k)}$$

$$a_l = \tilde{a}_{i,j}$$

이 방식은 큰 1차원 다항식 환 $\mathbb{Z}_q[x]/(x^N - 1)$을 다루기 쉬운 작은 두 개의 링의 Tensor Product 형태인 $(\mathbb{Z}_q[z]/(z^{2^k} - 1)) \otimes (\mathbb{Z}_q[y]/(y^h - 1))$로 decomposition하는 장점이 있다.

이 분해를 통해 N-point NTT를 수행하는 대신, h개의 2^k-point NTT와 2^k개의 h-point NTT로 쪼개서 병렬 처리가 가능해진다.

Write the coefficients $\tilde{a}_{i,j}$ into a matrix $\tilde{A} = (\tilde{a}_{i,j})_{h\times 2^k}$, and do h parallel 2^k-point NTT over $\mathbb{Z}_q[z]/(z^{2^k} - 1)$ with each row.

The corresponding point-wise multiplications are 2^k parallel degree-$(h-1)$ polynomial multiplications in the ring $\mathbb{Z}_q[y]/(y^h-1)$ with each column from $\tilde{A}$. Then we do h parallel 2^k-point INTT with each row. Denote the resulting matrix by $\tilde{C} = (\tilde{c}_{i,j})_{h \times 2^k}$.

Map $\sum_{i=0}^{h-1} \sum_{j=0}^{2^k-1} \tilde{c}_{i,j} y^i z^j$ back to $c = \sum_{l=0}^{h \cdot 2^k - 1} c_l x^l \in \mathbb{Z}_q[x]/(x^{h \cdot 2^k} - 1)$ according to the CRT formula

$$l = ((2^k)^{-1} \bmod h) \cdot 2^k \cdot i + (h^{-1} \bmod 2^k) \cdot h \cdot j \bmod h \cdot 2^k$$

to obtain $c_l x^l$ from $\tilde{c}_{i,j} y^i z^j$.

Def) Schönhage's trick

Map the multiplicand

$$a = \sum_{i=0}^{2mn-1} a_i x^i \in \mathbb{Z}_q[x]/(x^{2mn}-1)$$

$$\to \sum_{j=0}^{2n-1} \Big(\sum_{i=0}^{m-1} a_{m \cdot j + i} x^i \Big) y^j \in \big(\mathbb{Z}_q[x][y]/(y^{2n}-1) \big) / (x^m - y) \text{ with } y = x^m.$$

To compute multiplication in $(\mathbb{Z}_q[x][y]/(y^{2n}-1))/(x^m-y)$, one can first compute that in $\mathbb{Z}_q[x][y]/(y^{2n}-1)$, and then obtain the result modulo $(x^m - y)$.

And to compute multiplication in $\mathbb{Z}_q[x]$ with multiplicands of degree less than m, we can do it in $\mathbb{Z}_q[x]/(x^{2m}+1)$ without modulo $(x^{2m}+1)$.

Therefore, multiplication in $\mathbb{Z}_q[x][y]/(y^{2n}-1)$ can be computed in $(\mathbb{Z}_q[x]/(x^{2m}+1))[y]/(y^{2n}-1)$.

Note that it is an NTT-friendly ring and x is the primitive $4m$-th root of unity in $\mathbb{Z}_q[x]/(x^{2m}+1)$.

원리: 블록화

$$a(x) = a_0 + a_1 x + \cdots + a_{2mn-1} x^{2mn-1}$$

$x^m = y$라고 하고, 다항식 $a(x)$를 m개 항씩 블록으로 나눈다고 하자.
x의 지수 l을 m으로 나누어 몫(j)과 나머지(i)로 보면

$$l = m \cdot j + i \quad (0 \le i < m)$$

이므로

$$A(y) = \sum_{j=0}^{2n-1} \left(\sum_{i=0}^{m-1} a_{m \cdot j + i} x^i \right) y^j$$

이고 $A(y)$는 y에 대한 다항식이다.
이때 계수는 상수가 아닌, x에 대한 (차수 m 미만의) 다항식이다.
따라서 y에 대하여 $2n$-point NTT를 수행해야 하며, 계수끼리 덧셈/곱셈을 할 때 modulus $x^{2m}+1$으로 다항식 연산을 수행해야 한다.
이 기법은 NTT를 적용하고 싶은데 적절한 q가 없을 때 활용된다.
NTT를 하려면 N-th root of Unity)가 $\mathbb{Z}_q$ 안에 있어야 하지만, q가 작거나 조건이 안 맞으면 이러한 근이 존재하지 않을 수 있다.
Schönhage's Trick은 q를 찾는 대신 변수 x 자체를 root of Unity로 사용한다.
계산하는 ring의 modulus가 $x^{2m}+1$이므로, $x^{2m} \equiv -1$이고 따라서 $x^{4m} \equiv 1$이 된다.

즉, x라는 다항식 자체가 $4m$-th root of Unity 역할을 하므로,
$\mathbb{Z}_q$에 N-th root of unity가 없어도, ring을 확장함으로써 억지로 NTT가 가능하게 만든 것이다.
또한, Good's Trick은 $N = h \cdot 2^k$처럼 서로소인 경우에만 구조를 분해할 수 있는 반면, Schönhage's Trick은 서로소가 아니더라도 $N = m \cdot n$이기만 하면 언제든 ring과 연산 단위를 분해할 수 있다.

Nussbaumer's trick

Nussbaumer's trick is similar to Schönhage's trick.

It maps

$$a = \sum_{i=0}^{2mn-1} a_i x^i \in \mathbb{Z}_q[x]/(x^{2mn}+1)$$

$$\to \sum_{i=0}^{m-1} \Big(\sum_{j=0}^{2n-1} a_{m\cdot j+i} y^j \Big) x^i \in \big(\mathbb{Z}_q[y]/(y^{2n}+1)\big)\,[x]/(x^m - y) \;\text{ with }\; y = x^m.$$

To compute multiplication in $(\mathbb{Z}_q[y]/(y^{2n}+1))[x]/(x^m - y)$, first compute multiplication in $(\mathbb{Z}_q[y]/(y^{2n}+1))[x]$, and then obtain the result modulo $(x^m - y)$.

And to compute multiplication in $(\mathbb{Z}_q[y]/(y^{2n}+1))[x]$ with multiplicands of degree less than n, one can do it in $(\mathbb{Z}_q[y]/(y^{2n}+1))[x]/(x^{2n}-1)$ for $n \geq m$ without modulo $(x^{2n}-1)$.

Note that it is an NTT-friendly ring and y is the primitive $4n$-th root of unity in $\mathbb{Z}_q[y]/(y^{2n}+1)$.

원리: 축 변환
이 기법은 두 단계의 축 변환을 연속적으로 수행한다.
먼저 Schönhage와 동일하게, 원래 다항식 a가 $\mathbb{Z}_q[x]/(x^{2mn}+1)$에 있을 때 이를 m개 항 단위로 나눠 $y = x^m$으로 치환하여 2차원 형태로 만든다.

$$a = \sum_{l=0}^{2mn-1} a_l x^l \;\mapsto\; \sum_{j=0}^{2n-1} \left(\sum_{i=0}^{m-1} a_{m\cdot j+i} x^i \right) y^j \in (\mathbb{Z}_q[x]/(x^m - y))[y]/(y^{2n}+1)$$

결과적으로, y에 대한 다항식이 되며, 계수는 x에 대한 다항식이 된다.
최종적으로 얻고자 하는 ring은 $\mathbb{Z}_q[y]/(y^{2n}+1)$이 계수가 되고,
x가 변수가 되는 ring $(\mathbb{Z}_q[y]/(y^{2n}+1))[x]/(x^m - y)$이다.

$$\sum_{j=0}^{2n-1} \underbrace{\left(\sum_{i=0}^{m-1} a_{mj+i} x^i \right)}_{\text{Coefficient } C_j(x)} y^j \;=\; \sum_{i=0}^{m-1} \underbrace{\left(\sum_{j=0}^{2n-1} a_{mj+i} y^j \right)}_{\text{Coefficient } D_i(y)} x^i$$

이 ring은 x의 차수가 m 미만, y의 차수가 $2n$ 미만이며, y가 $4n$-th root of unity 역할을 하게 된다.
Nussbaumer's Trick은 계산의 효율성 극대화한다.
이 기법은 $\mathbb{Z}_q$에 N-th root of unity가 없을 때도 변수 y나 x 자체를 사용하여 NTT와 유사한 고속 변환(Polynomial Transform)을 강제로 적용할 수 있게 한다.
또한, 전체 $2mn$-차수 다항식의 곱셈을 y에 대한 $2n$-point NTT로 바꾸며, 이 과정에서 $2n$개의 작은 곱셈을 병렬로 수행하게 된다.
가장 큰 이점은 작은 곱셈을 할 때 modulo $(x^{2m}+1)$ 대신 modulo $(x^{2n}-1)$과 같은 단순한 modulo를 사용할 수 있게 변환한다는 점이다.

2.5 [+] NUMBER THEORETIC TRANSFORM OVER $\mathbb{Z}_Q[X]/(X^N - x^{N/2} + 1)$

In this section, introduce some progresses about relaxing the requirement of n being power-of-two such that NTT can be utilized over non-power-of-two rings.

2.5.1 Incomplete FFT trick over $\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1)$

Some progress about relaxing the requirement of n being a power-of-two is made by Lyubashevsky and Seiler.

They first introduced a special incomplete NTT to lattice-based cryptographic schemes,

by offering a new non-power-of-two ring structure $\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1)$ where $n = 3 \cdot 2^e$

instead of a power of two and q is a prime number satisfying $q \equiv 1 \pmod{n}$ such that ψ_n exits.

Actually, $x^n - x^{n/2} + 1$ is the $3n$-th cyclotomic polynomial of degree n, where $n = 3^l \cdot 2^e$ $(l \geq 0, e \geq 1)$, not a power-of-two cyclotomic polynomial anymore.

$$\Phi_{3n}(x) = x^n - x^{n/2} + 1$$

$$\Phi_{3n}(x) = \Phi_{6(n/2)}(x) = \Phi_6(x^{n/2}) = (x^{n/2})^2 - (x^{n/2}) + 1$$

$$\therefore \Phi_{3n}(x) = x^n - x^{n/2} + 1$$

이때 $\Phi_{3n}(x)$의 두 근의 합은 $-(-1)/1 = 1$, 곱은 $1/1 = 1$.
ζ_1의 oreder가 $3^l \cdot 2^e$ 꼴이어야 하므로 $\zeta_1 = \psi_n^{n/6}$ 이라 하고,
order가 최소인 6일 때

$$\zeta_1^6 = 1 \implies \zeta_1 \cdot \zeta_2 = \zeta_1 \cdot \zeta_1^5 = 1$$

이므로 나머지 하나의 근은 $\zeta_2 = \zeta_1^5$ 라 한다.

The main observation they use is the CRT map, as follows

$$\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1) \cong \mathbb{Z}_q[x]/(x^{n/2} - \zeta_1) \times \mathbb{Z}_q[x]/(x^{n/2} - \zeta_2),$$

$$\text{where} \quad \zeta_1 + \zeta_2 = 1 \text{ and } \zeta_1 \cdot \zeta_2 = 1.$$

In their instantiation, they choose $\zeta_1 = \psi_n^{n/6}$ and $\zeta_2 = \zeta_1^5$.

As for its forward transform, $\boldsymbol{a} \in \mathbb{Z}q[x]/(x^n - x^{n/2} + 1)$ from the 0-th level generates its images

$$\boldsymbol{a}_l = \boldsymbol{a} \bmod x^{n/2} - \zeta_1 \in \mathbb{Z}q[x]/(x^{n/2} - \zeta_1)$$

and

$$\boldsymbol{a}_r = \boldsymbol{a} \bmod x^{n/2} - (1 - \zeta_1) \in \mathbb{Z}q[x]/(x^{n/2} - \zeta_2)$$

respectively in the first level, by using the fact that $\zeta_2 = 1 - \zeta_1$. In order to get the coefficients, one can compute

$$a_{l,i} = a_i + \zeta_1 a_{i+n/2}, \quad a_{r,i} = a_i + a_{i+n/2} - \zeta_1 a_{i+n/2} \quad (i = 0, 1, \ldots, n/2 - 1).$$

Different from the radix-2 Cooley-Tukey algorithm, there are extra $n/2$ additions in this case.

These additional additions don't cost much.

For a fast NTT algorithm, one can continue with a similar radix-2 ($\log \frac{n}{3}$)-level FFT trick in $\mathbb{Z}_q[x]/(x^{n/2} - \zeta_1)$ and $\mathbb{Z}_q[x]/(x^{n/2} - \zeta_2)$, as in the power-of-two cyclotomic rings above, until the leaf nodes are of the form $\mathbb{Z}_q[x]/(x^3 - \psi_n^j)$ instead of linear terms.

The inverse transform can be obtained by inverting the trick mentioned above, where Gentleman-Sande butterflies are used in the radix-2 steps.

The point-wise multiplication is performed about the corresponding polynomials of degree 2 in each $\mathbb{Z}_q[x]/(x^3 - \psi_n^j)$. Detailedly, the CRT map can be described as follows

$$\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1) \;\cong\; \prod_{j \in \mathbb{Z}_n^\times} \mathbb{Z}_q[x]/(x^3 - \psi_n^j),$$

where $\mathbb{Z}_n^\times$ is the group of invertible elements of $\mathbb{Z}_n$.

2.5.2 Splitting Polynomial Ring over $\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1)$

The method based on splitting the polynomial ring can be generalized to the ring $\mathbb{Z}_q[x]/(x^n - x^{n/2} + 1)$ where $n = 3 \cdot 2^e$ and q is a prime number, based on which Liang et al. proposed a generalized, modular, and parallelizable NTT method referred to as Generalized 3-NTT (G3-NTT for simplicity).

Similarly, let α, β be non-negative integers. The general α-round G3-NTT with β levels cropped is essentially based on the following isomorphism.

$$\Psi_{\alpha,3} : \mathbb{Z}_q[x]/(x^n - x^{n/2} + 1) \;\cong\; \left(\mathbb{Z}_q[y]/(y^{\frac{n}{3 \cdot 2^\alpha}} - y^{\frac{n}{3 \cdot 2^{\alpha+1}}} + 1)\right)[x]/(x^{3 \cdot 2^\alpha} - y)$$

$$\boldsymbol{a} = \sum_{i=0}^{n-1} a_i x^i \;\mapsto\; \Psi_{\alpha,3}(\boldsymbol{a}) = \sum_{i=0}^{3 \cdot 2^\alpha - 1} \left(\sum_{j=0}^{(n/3 \cdot 2^\alpha) - 1} a_{3 \cdot 2^\alpha \cdot j + i} y^j \right) x^i$$

where $y^{n/3 \cdot 2^\alpha} - y^{n/3 \cdot 2^{\alpha+1}} + 1$ is the $n/2^\alpha$-th cyclotomic polynomial of degree $n/3 \cdot 2^\alpha$.

Similar to NTRU, there is a CRT map as follows

$$\mathbb{Z}_q[y]/(y^{n/3 \cdot 2^\alpha} - y^{n/3 \cdot 2^{\alpha+1}} + 1) \;\cong\; \mathbb{Z}_q[y]/(y^{n/3 \cdot 2^{\alpha+1}} - \zeta_1) \times \mathbb{Z}_q[y]/(y^{n/3 \cdot 2^{\alpha+1}} - \zeta_2)$$

where $\zeta_1 + \zeta_2 = 1,\ \zeta_1 \cdot \zeta_2 = 1$.

It turns out that radix-2 truncated-NTTs can be performed in $\mathbb{Z}_q[y]/(y^{n/3 \cdot 2^{\alpha+1}} - \zeta_1)$ and $\mathbb{Z}_q[y]/(y^{n/3 \cdot 2^{\alpha+1}} - \zeta_2)$.

If there are β levels to be cropped, $\beta = 0, 1, \ldots, \log \frac{n}{3 \cdot 2^\alpha} - 1$, the modulus q can be chosen to satisfy only $q \equiv 1 \bmod n/2\alpha + \beta$ such that the primitive $n/2^{\alpha+\beta}$-th root of unity $\psi_{n/2^{\alpha+\beta}}$ exits.

The leaf nodes of the CRT tree map are degree-2^β polynomials, e.g., $\mathbb{Z}_q[y]/(y^{2^\beta} - \psi^j_{n/2^{\alpha+\beta}})$.

They choose $\zeta_1 = \psi^{n/(6 \cdot 2^{\alpha+\beta})}_{n/2^{\alpha+\beta}}$ and $\zeta_2 = \zeta_1^5$.

One-iteration Karatsuba algorithm can be used in a same way as H-NTT.

Given appropriate and fixed (n, q), the computational complexity of G3-NTT can reach its optimization if $\alpha = 0, \beta = 0$.

3 BASIC RADIX-2 FAST NUMBER THEORETIC TRANSFORM

Here, "radix-2" means the length n of NTT has a factor as a power of two, resulting that original algorithm can be divided into two parts of less length.

Transforms	Cooley-Tukey algorithm	Gentleman-Sande algorithm
—	—	—
NTT	$\text{NTT}^{CT}_{bo\to no}, \text{NTT}^{CT}_{no\to bo}$	$\text{NTT}^{GS}_{bo\to no}, \text{NTT}^{GS}_{no\to bo}$
INTT	$\text{INTT}^{CT}_{bo\to no}, \text{INTT}^{CT}_{no\to bo}$	$\text{INTT}^{GS}_{bo\to no}, \text{INTT}^{GS}_{no\to bo}$
NTT^{ψ}	$\text{NTT}^{CT,\psi}_{bo\to no}, \text{NTT}^{CT,\psi}_{no\to bo}$	—
$\text{INTT}^{\psi^{-1}}$	—	$\text{INTT}^{GS,\psi^{-1}}_{bo\to no}, \text{INTT}^{GS,\psi^{-1}}_{no\to bo}$

3.1 FFT Trick

Thm. Chinese Remainder Theorem in ring form

Let R be a commutative ring with multiplicative identity,

$I_1, I_2, \ldots, I_k$ be ideals in R that are pairwise co-prime, and I be their intersection.

Then there is a ring isomorphism

$$\Phi : R/I \cong R/I_1 \times R/I_2 \times \cdots \times R/I_k.$$

FFT trick means that according to Theorem,

for polynomial rings $\mathbb{Z}_q[x]/(x^{2m} - \omega^2)$ where $m > 0$ and invertible $\omega \in \mathbb{Z}_q$,

$$\Phi : \mathbb{Z}_q[x]/(x^{2m} - \omega^2) \;\cong\; \mathbb{Z}_q[x]/(x^m - \omega) \times \mathbb{Z}_q[x]/(x^m + \omega)$$

$$\boldsymbol{a} \mapsto (\boldsymbol{a}' = \boldsymbol{a} \bmod x^m - \omega, \boldsymbol{a}'' = \boldsymbol{a} \bmod x^m + \omega)$$

and the detailed mapping process

$$\Phi\left(\sum_{i=0}^{2m-1} a_i x^i\right) = \left(\sum_{i=0}^{m-1} (a_i + \omega \cdot a_{i+m}) x^i, \quad \sum_{i=0}^{m-1} (a_i - \omega \cdot a_{i+m}) x^i\right)$$

$$\Phi^{-1}\left(\sum_{i=0}^{m-1} a'_i x^i, \quad \sum_{i=0}^{m-1} a''_i x^i\right) = \sum_{i=0}^{m-1} \frac{1}{2}(a'_i + a''_i) x^i + \sum_{i=0}^{m-1} \frac{\omega^{-1}}{2}(a'_i - a''_i) x^{i+m}.$$

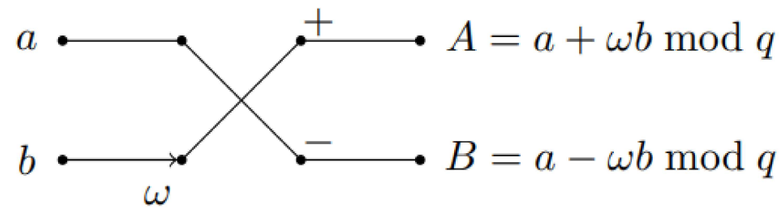
As for the forward FFT trick, it is very effective to compute $\boldsymbol{a}'$ and $\boldsymbol{a}''$.

Their i-th coefficient can be computed via

$$a'_i = a_i + \omega \cdot a_{i+m}, \quad a''_i = a_i - \omega \cdot a_{i+m},$$

where $\omega \cdot a_{i+m}$ is computed once but used twice, $i = 0, 1, \ldots, m-1$.

This type of operation is known as the Cooley-Tukey butterfly or CT butterfly Multiplication → Addition for short.



(a) Cooley-Tukey butterfly

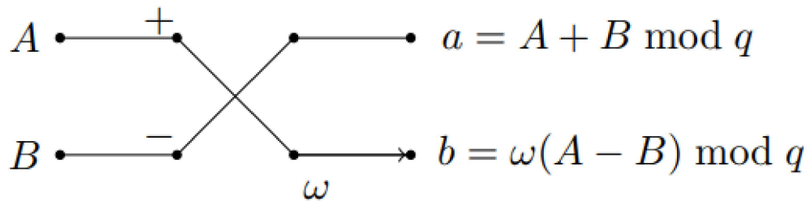
The forward FFT trick totally takes m multiplications, m additions, and m subtractions.

As for the inverse FFT trick, the i-th and $(i + \frac{n}{2})$-th coefficient of $\boldsymbol{a}$ can be derived from the i-th coefficient of $\boldsymbol{a}'$ and $\boldsymbol{a}''$.

$$a_i = (a'_i + a''_i)/2, \quad a_{i+m} = \omega^{-1}(a'_i - a''_i)/2 \quad (i = 0, 1, \ldots, m-1).$$

In the practical applications, the scale factor 2 can be omitted, with multiplying a total factor in the end.

This type of operation is known as Gentlemen-Sande butterfly or GS butterfly Addition → Multiplication for short.



(b) Gentleman-Sande butterfly

Based on the FFT trick, introduce the fast algorithms to compute NTT and INTT over $\mathbb{Z}_q[x]/(x^n - 1)$, as well as NTT^{ψ} and $\text{INTT}^{\psi^{-1}}$ over $\mathbb{Z}_q[x]/(x^n + 1)$ in the subsequent two subsections.

3.2 FFT Trick for CC-based NTT over $\mathbb{Z}_q[x]/(x^n - 1)$

How to use the FFT trick to compute NTT and INTT over $\mathbb{Z}_q[x]/(x^n - 1)$,

where n is a power of two and q is a prime number satisfying $q \equiv 1 \pmod{n}$.

Denote by ω_n the primitive n-th root of unity in $\mathbb{Z}_q$.

$$\mathbb{Z}_q[x]/(x^n - 1) \cong \mathbb{Z}_q[x]/(x^{\frac{n}{2}} - 1) \times \mathbb{Z}_q[x]/(x^{\frac{n}{2}} + 1)$$

Notice that the forward FFT trick can be applied repeatedly to map $\mathbb{Z}_q[x]/(x^{\frac{n}{2}} \pm 1)$, according to the fact $x^{\frac{n}{2}} + 1 = x^{\frac{n}{2}} - \omega_n^{\frac{n}{2}}$.

In fact, $x^n - 1$ has n distinct roots in $\mathbb{Z}_q$, i.e., ω_n^i $(i = 0, 1, \ldots, n-1)$.

Therefore, forward FFT trick can be applied recursively from $\mathbb{Z}_q[x]/(x^n - 1)$ all the way down to linear terms,

$$\mathbb{Z}_q[x]/(x^n - 1) \cong \prod_{i=0}^{n-1} \mathbb{Z}_q[x]/(x - \omega_n^{\text{brv}_n(i)}).$$

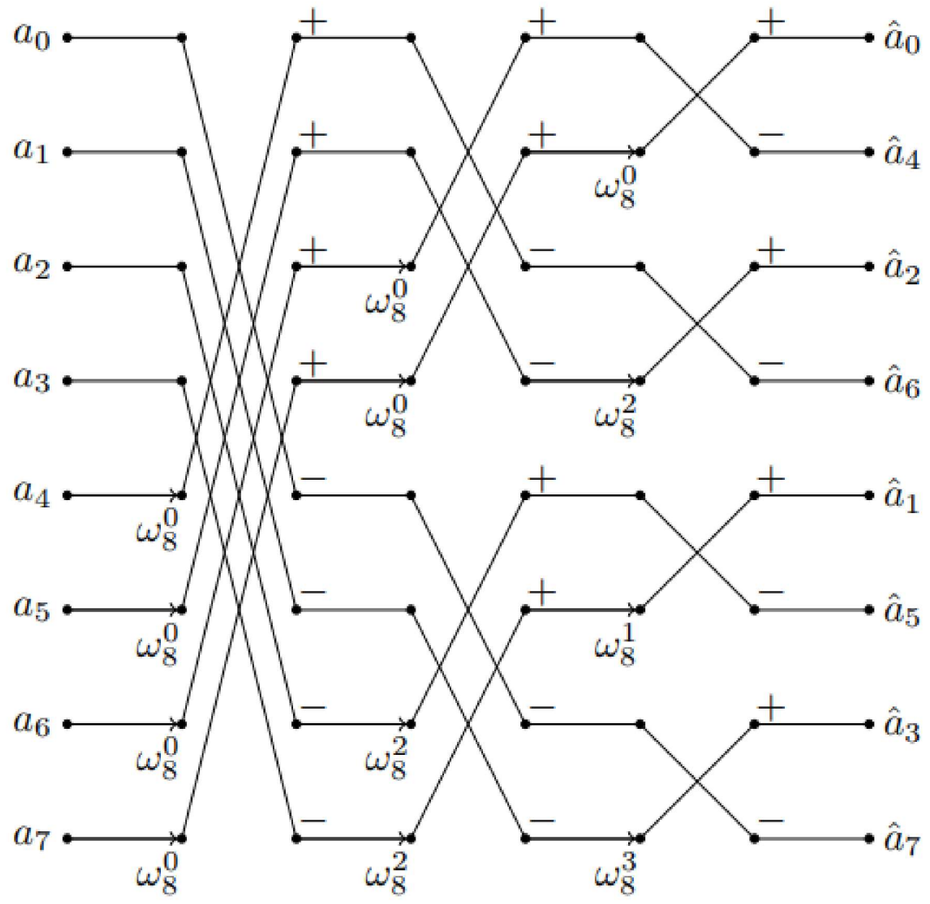
Finally, $\boldsymbol{a}$ generates its images in $\mathbb{Z}_q[x]/(x - \omega_n^{\text{brv}_n(i)})$, i.e., $\hat{a}_{\text{brv}_n(i)}$, which turns out to be the coefficient of $\hat{\boldsymbol{a}}$ indexed by $\text{brv}_n(i)$.

Notice that using the Cooley-Tukey algorithm, the coefficients of the input polynomials are indexed under natural order, while the coefficients of the output polynomials are indexed under bit-reversed order.

This Cooley-Tukey NTT algorithm is denoted by $\text{NTT}^{CT}_{no\rightarrow bo}$,

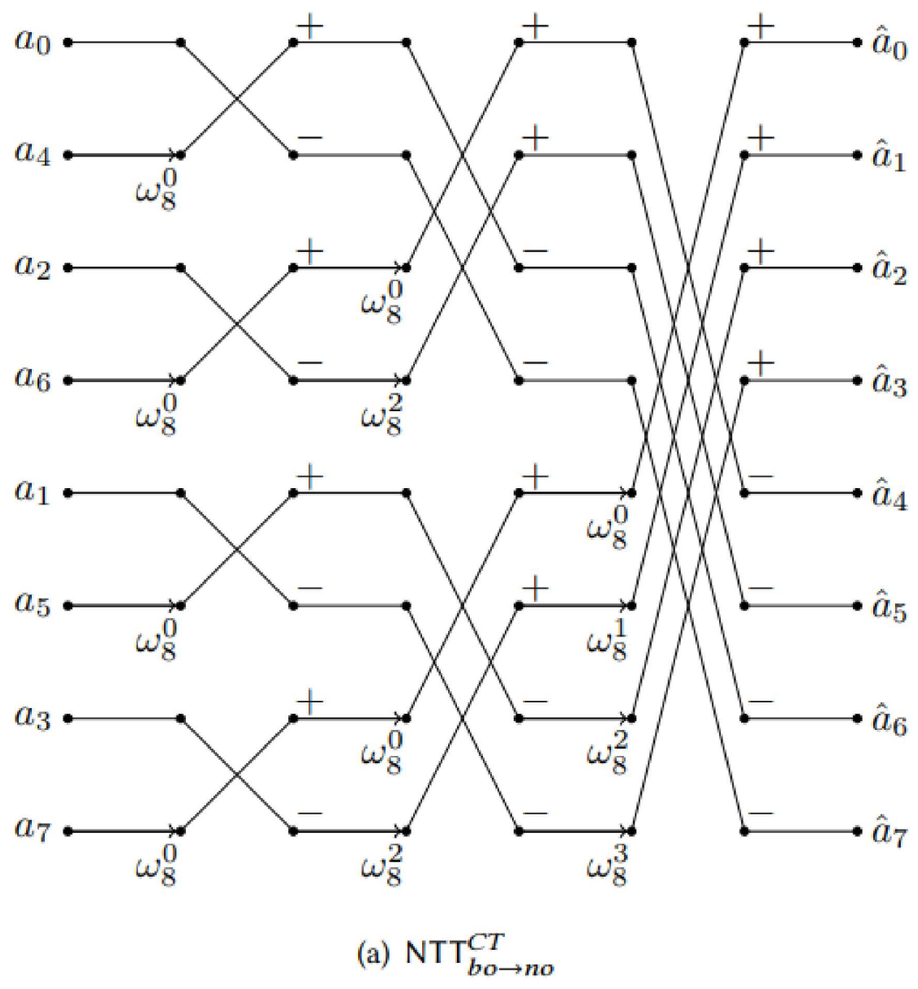
where the subscripts $no \rightarrow bo$ indicate that the input coefficients are under natural order and the output coefficients are under bit-reversed order.

The signal flow of $\text{NTT}^{CT}_{no\rightarrow bo}$ for $n = 8$ can be seen in Figure (c).



(c) $\text{NTT}^{CT}_{no\rightarrow bo}$

Adjust the input to bit-reversed order, then the Cooley-Tukey butterflies in the signal flow are changed elsewhere, as in Figure (a). The output will be under natural order. This type of NTT is denoted by $\text{NTT}^{CT}_{bo\rightarrow no}$.



(a) $\text{NTT}^{CT}_{bo\rightarrow no}$

Figure 2 shows the detailed process of using FFT trick to map $\mathbb{Z}_q[x]/(x^n - 1)$.

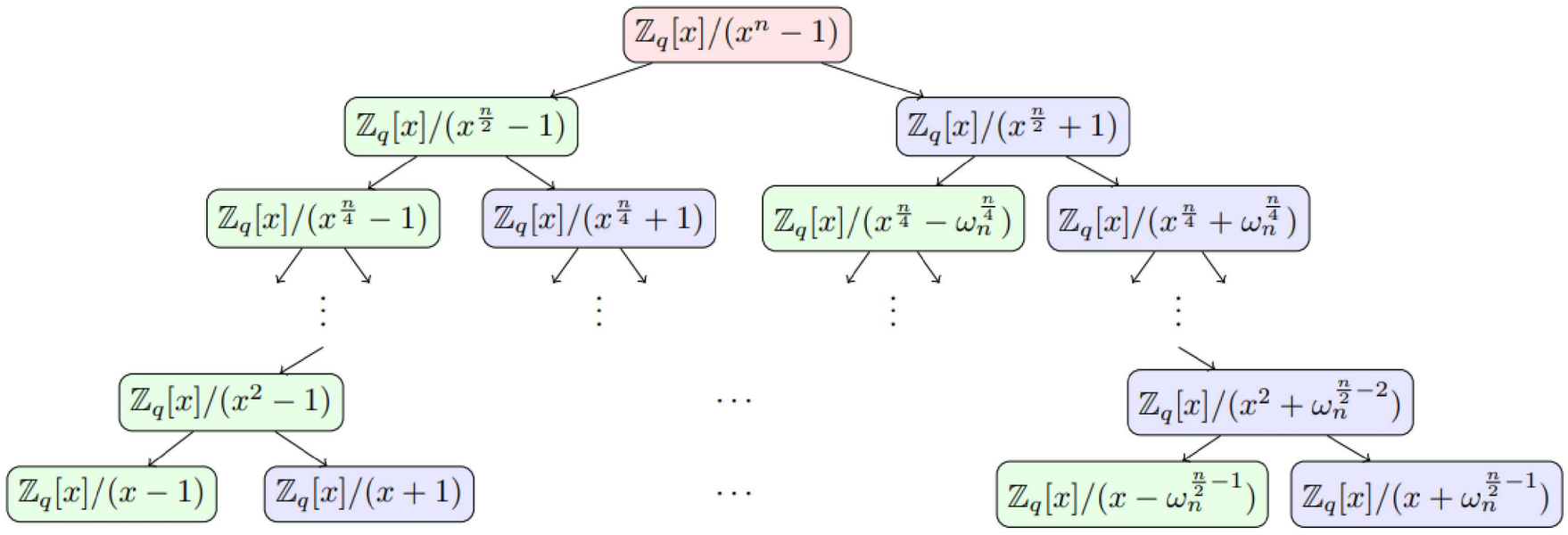


Fig. 2. CRT map of FFT trick over $\mathbb{Z}_q[x]/(x^n-1)$

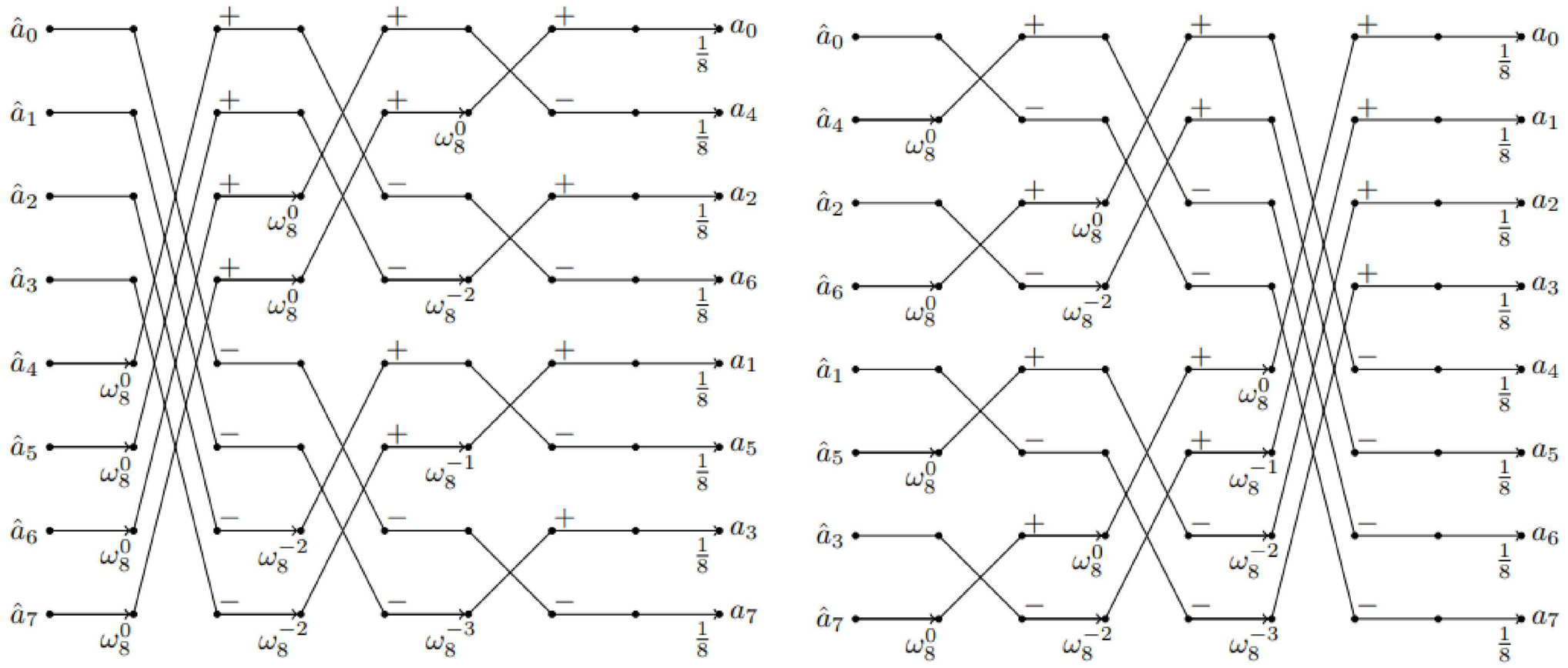
The mapping process takes on the shape of a binary tree, with the root node being the 0-th level and the leaf nodes being the $(\log n)$-th level. $(x^n-1) \to (x-1)*(x+1)*\ldots*(x-w^{(n/2)-1}*(x+w^{(n/2)-1})$

After the k-th level, $0 \leq k < \log n$, there are 2^{k+1} nodes.

The fast algorithm to compute INTT can be obtained by iteratively inverting the CRT mappings.

In this case, the coefficients of the input polynomials are indexed under bit-reversed order, i.e., $\hat{a}_{\mathrm{brv}_n(i)}$ $(i = 0, 1, \ldots, n-1)$.

Apply the inverse FFT trick to the computation from the $(k+1)$-th level to the k-th level, where $1 \leq k < \log n$. $(x-1)*(x+1)*\ldots*(x-w^{(n/2)-1}*(x+w^{(n/2)-1}) \to (x^n-1)$

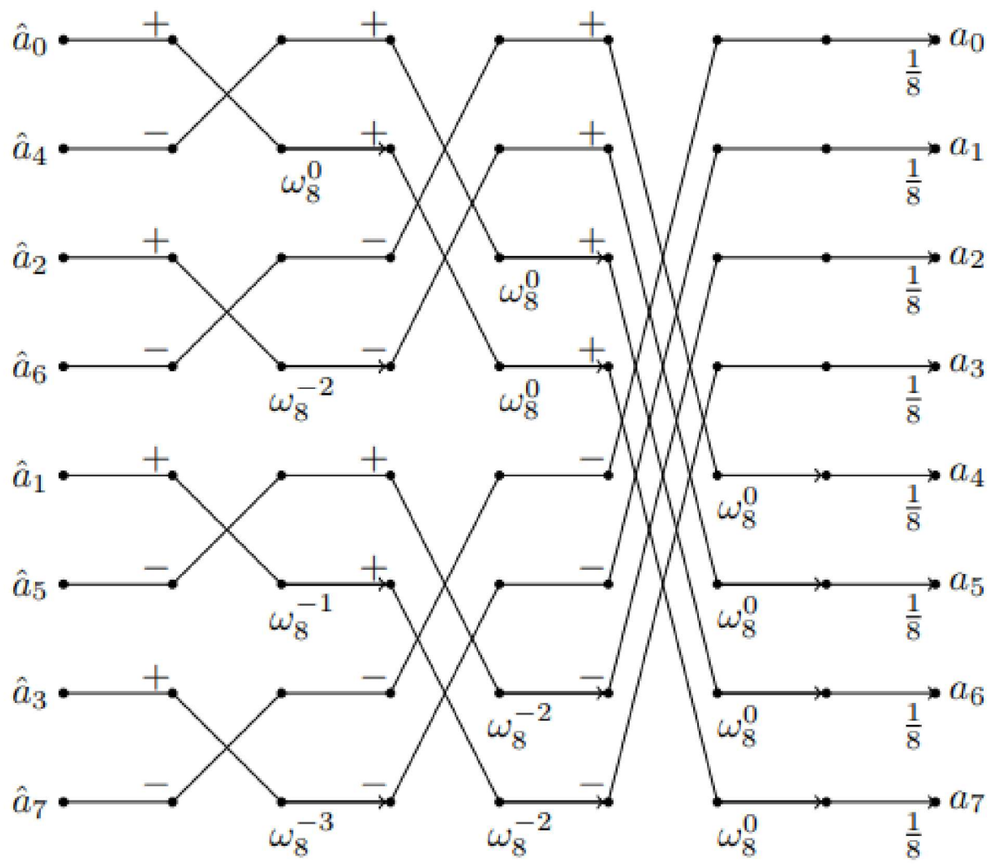


(c) $\mathrm{INTT}^{CT}_{no\to bo}$ (a) $\mathrm{INTT}^{CT}_{bo\to no}$

Note that the scale factor 2 in each level of the Gentleman-Sande butterfly can be omitted, with multiplying the final result by n^{-1} in the end.

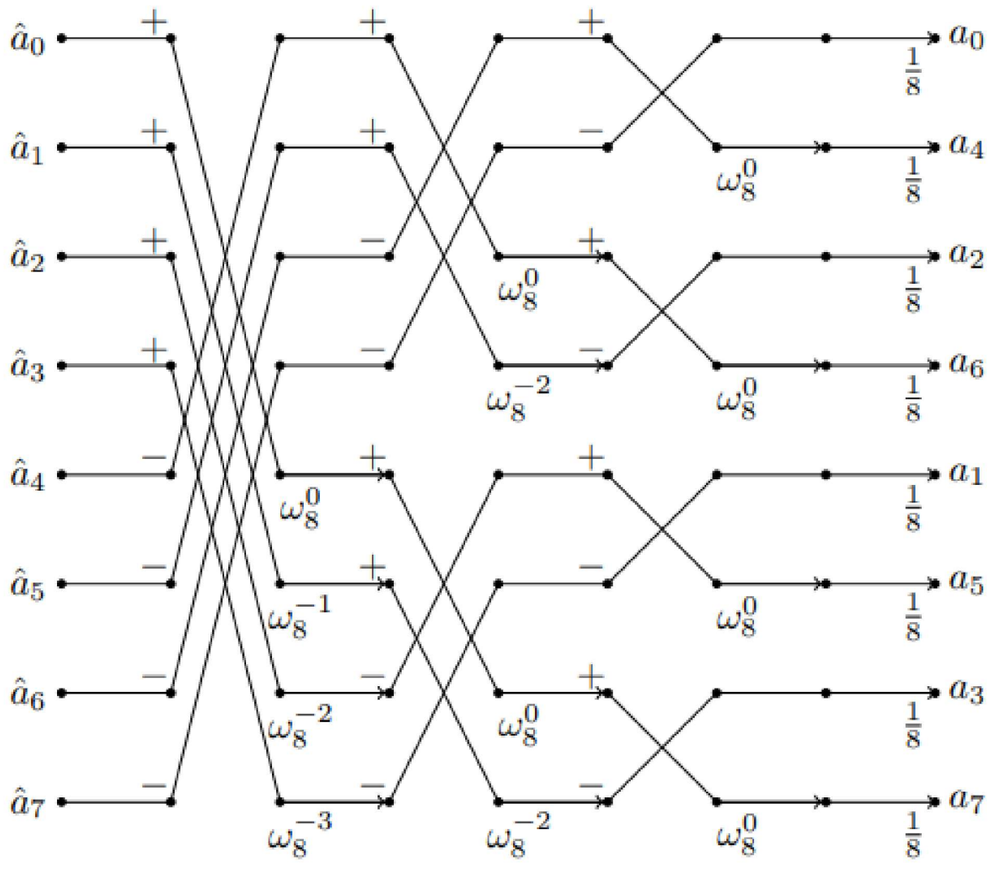
The coefficients of the output polynomials are indexed under natural order.

This type of Gentleman-Sande INTT algorithm is denoted by $\mathrm{INTT}^{GS}_{bo\to no}$.



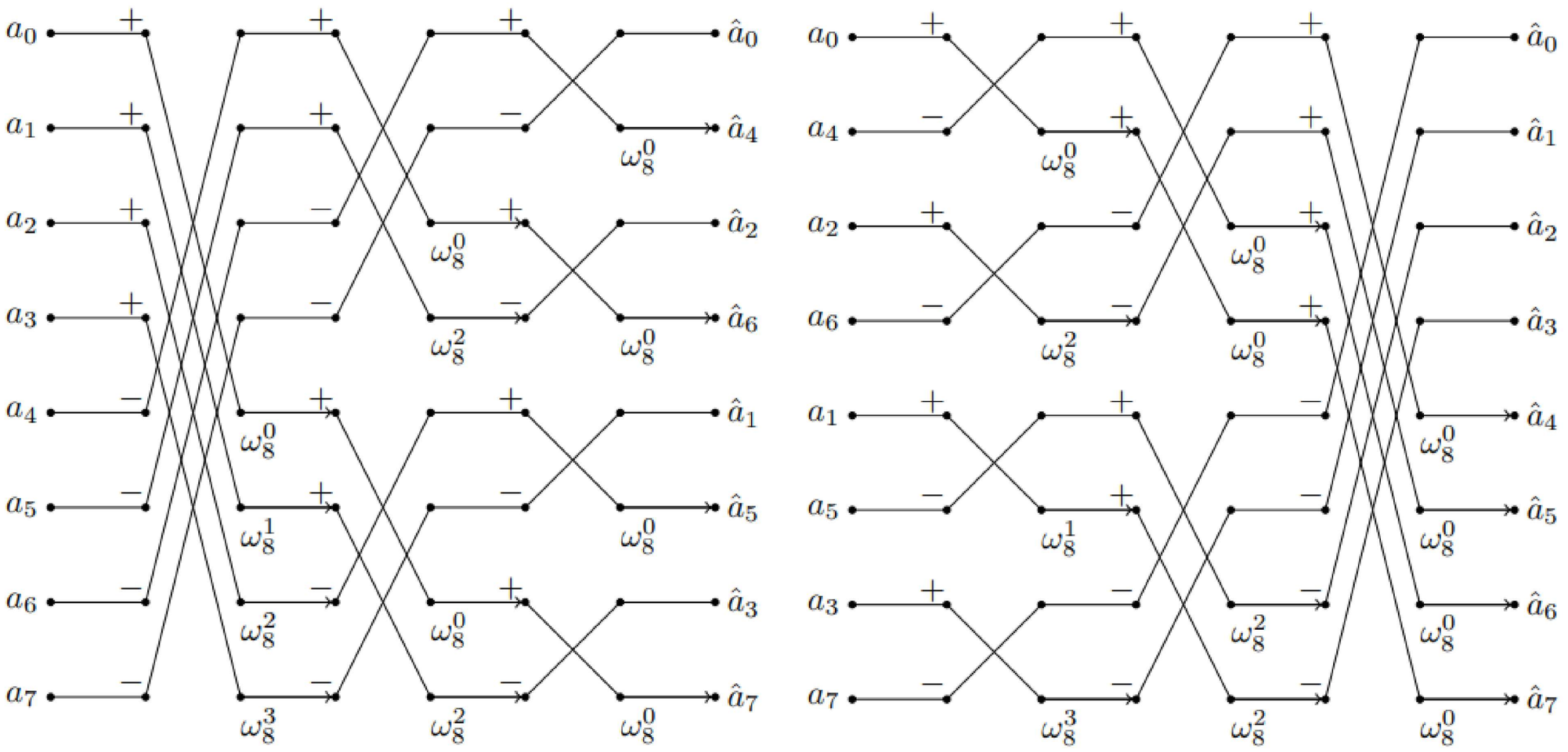
(b) $\mathrm{INTT}^{GS}_{bo\rightarrow no}$

Adjust its input/output order and we can obtain $\mathrm{INTT}^{GS}_{no\rightarrow bo}$.



(d) $\mathrm{INTT}^{GS}_{no\rightarrow bo}$

- NTT^{GS}



(d) $\mathrm{NTT}^{GS}_{no\to bo}$ (b) $\mathrm{NTT}^{GS}_{bo\to no}$

3.3 FFT Trick for NWC-based NTT over $\mathbb{Z}_q[x]/(x^n+1)$

The FFT trick to compute NTT^{ψ} and $\mathrm{INTT}^{\psi^{-1}}$ over $\mathbb{Z}_q[x]/(x^n+1)$,

where n is a power of two and q is a prime number satisfying $q \equiv 1 \pmod{2n}$.

Since $\psi_{2n}^{n/2} \equiv -1 \pmod{q}$, it holds that

$$x^n + 1 = x^n - \psi_{2n}^{n/2} = (x^{n/2} - \psi_{2n}^{n/2})(x^{n/2} + \psi_{2n}^{n/2}) \pmod{q}.$$

As for NTT^{ψ}, the forward FFT trick implies that we have the following isomorphism

$$\mathbb{Z}_q[x]/(x^n+1) \;\cong\; \mathbb{Z}_q[x]/(x^{\frac{n}{2}} - \psi_{2n}^{\frac{n}{2}}) \times \mathbb{Z}_q[x]/(x^{\frac{n}{2}} + \psi_{2n}^{\frac{n}{2}})$$

FFT trick can be applied repeatedly.

Notice that $x^n + 1$ has n distinct roots in $\mathbb{Z}_q$, i.e., $\psi_{2n}^{2i+1}, i = 0, 1, \ldots, n-1$.

Therefore, there is a CRT isomorphism similarly.

$$\mathbb{Z}_q[x]/(x^n+1) \;\cong\; \prod_{i=0}^{n-1} \mathbb{Z}_q[x]/(x - \psi_{2n}^{2\mathrm{brv}_n(i)+1}).$$

Figure 3 shows the detailed process of using FFT trick to map $\mathbb{Z}_q[x]/(x^n+1)$.

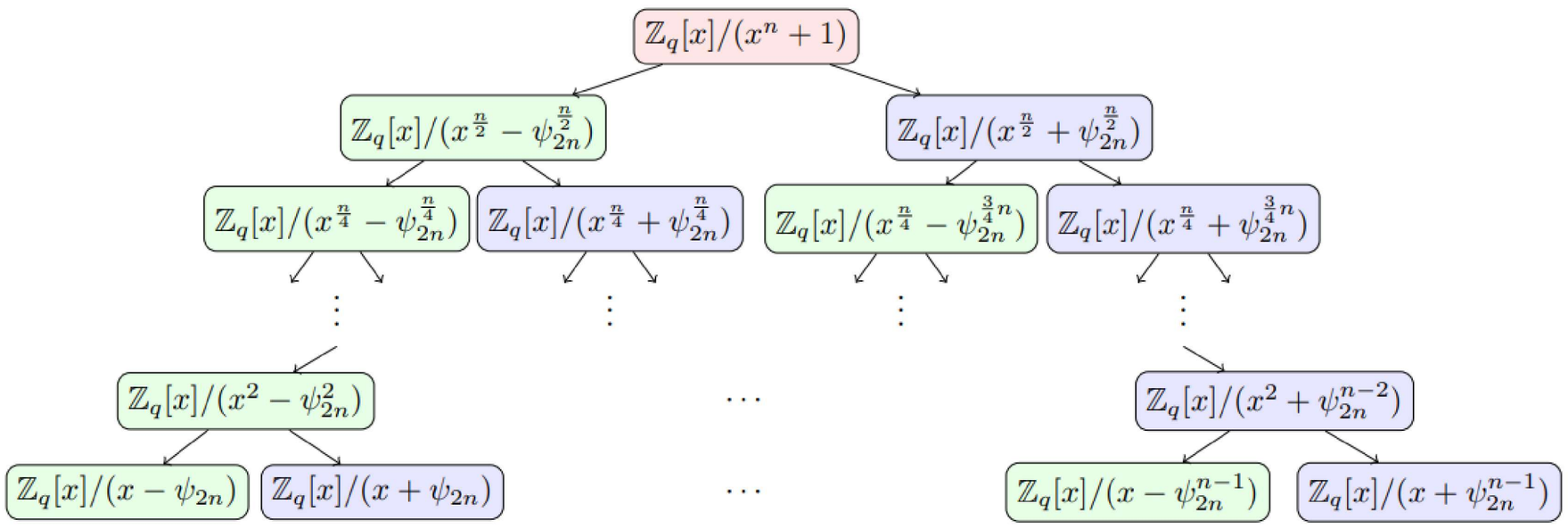


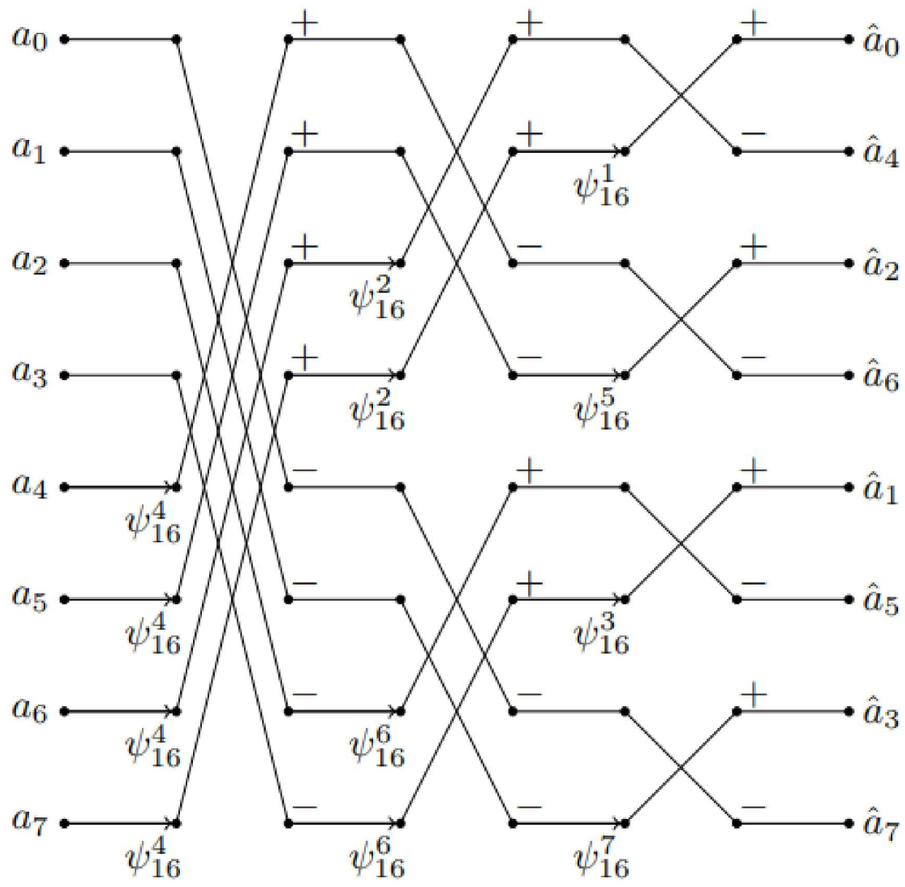
Fig. 3. CRT map of FFT trick over $\mathbb{Z}_q[x]/(x^n+1)$

After the k-th level, where $0 \leq k < \log n$, it produces

$$\mathbb{Z}_q[x]/(x^{n/2^{k+1}} \pm \psi_{2n}^{\mathrm{brv}_n(2^k+i)}) \qquad (i = 0, 1, \ldots, 2^k - 1)$$

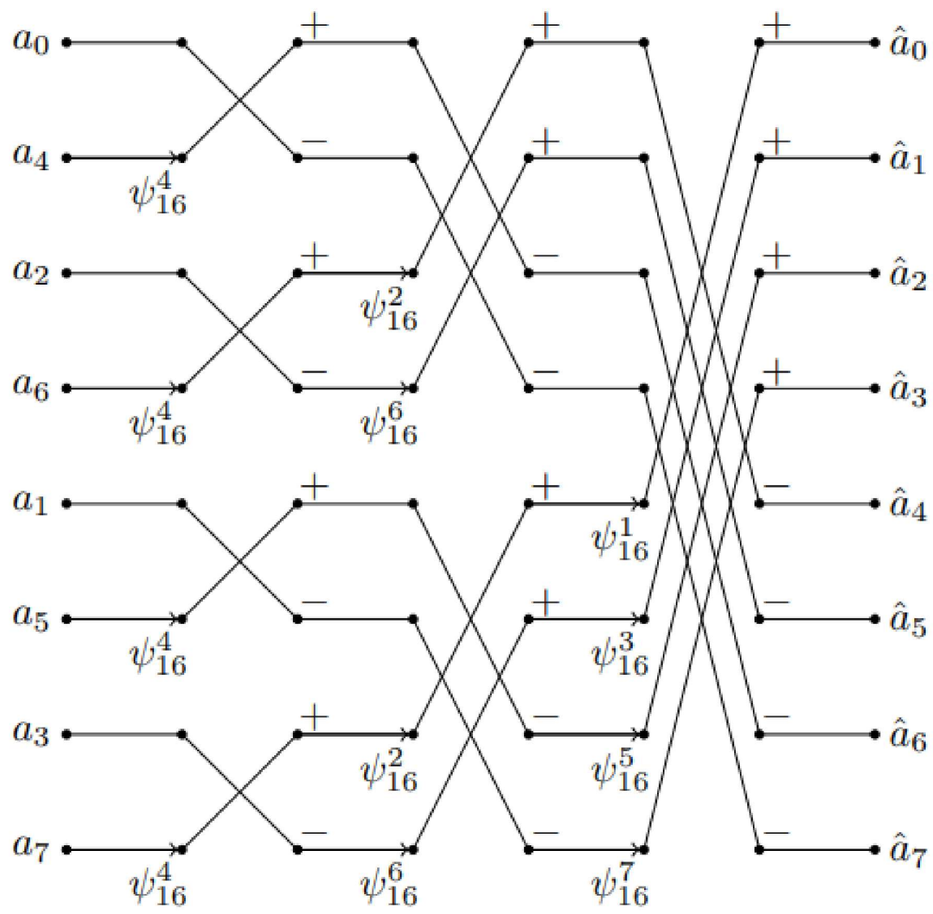
with pairs of rings.

Such fast algorithm of NTT^{ψ} is denoted by $\mathrm{NTT}^{CT,\psi}_{no \to bo}$.



(c) $\mathrm{NTT}^{CT,\psi}_{no \to bo}$

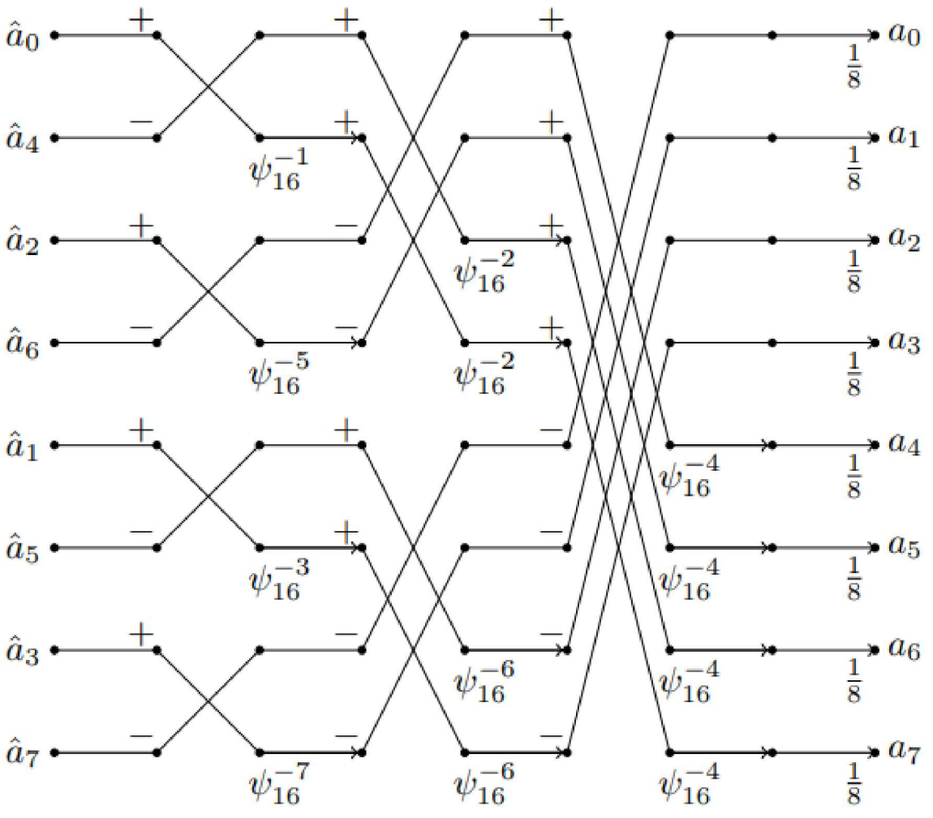
Similarly, by adjusting its input/output order, we can get $\mathrm{NTT}^{CT,\psi}_{bo \to no}$.



(a) $\mathrm{NTT}^{CT,\psi}_{bo\rightarrow no}$

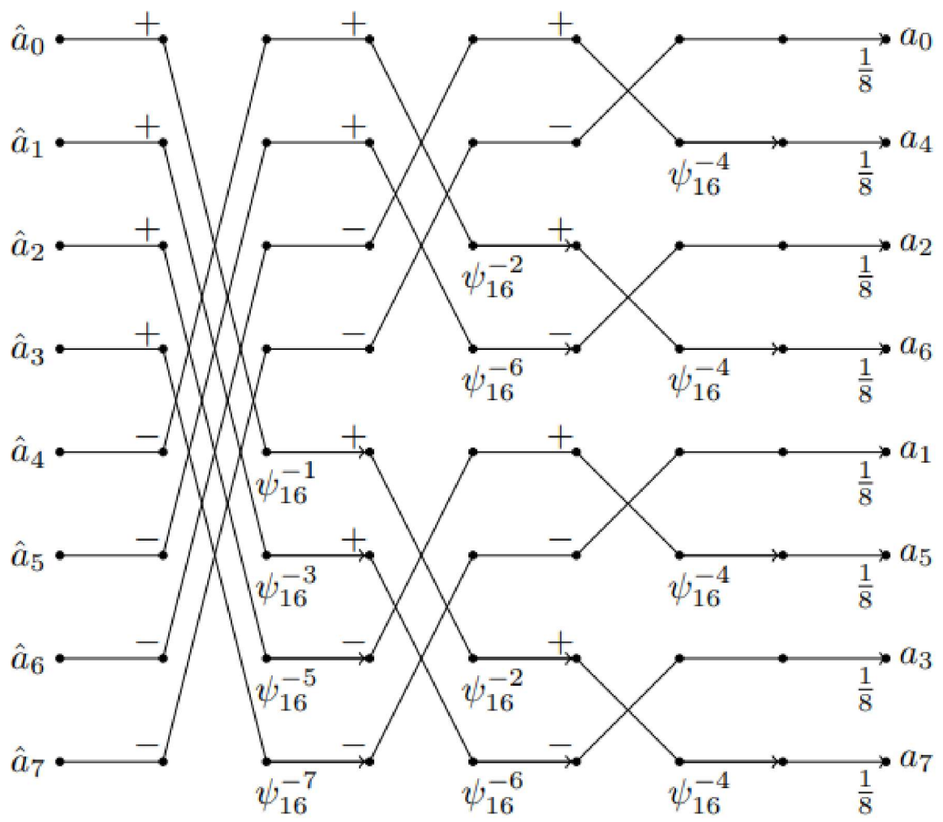
Similarly, the fast algorithm to compute $\mathrm{INTT}^{\psi^{-1}}$ can be obtained by iteratively inverting the CRT mapping process.

This type of fast algorithm for $\mathrm{INTT}^{\psi^{-1}}$ is denoted by $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\rightarrow no}$.



(b) $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\rightarrow no}$

Adjust the input/output order and get $\mathrm{INTT}^{GS,\psi^{-1}}_{no\rightarrow bo}$.



(d) $\text{INTT}_{no\to bo}^{GS,\psi^{-1}}$

Omitting the scale factor 2 in each level can multiply the final result by n^{-1} in the end.

There is an alternative way to deal with the total factor n^{-1}.

Zhang et al. noticed that n^{-1} can both be integrated into the computing process of each level, based on the fact that the scale factor 2 will be dealt with directly, by using addition and displacement shifting to compute $x/2 \bmod q$.

$$x/2 \equiv \begin{cases} (x \gg 1) \mod q, & x \text{ is even} \\ (x \gg 1) + (q+1)/2 \mod q, & x \text{ is odd} \end{cases}$$

3.4 Twisted FFT Trick

The Gentleman-Sande algorithm can be applied to compute NTT,

and the Cooley-Tukey algorithm can be applied to compute INTT.

It means, mapping

$$\mathbb{Z}_q[x]/(x^n - 1) \;\cong\; \mathbb{Z}_q[x]/(x^{\frac{n}{2}} - 1) \times \mathbb{Z}_q[x]/(x^{\frac{n}{2}} + 1)$$

by CRT, followed by mapping $\mathbb{Z}_q[x]/(x^{n/2} + 1)$ via following isomorphism

$$\Psi : \mathbb{Z}_q[x]/(x^{n/2} + 1) \;\cong\; \mathbb{Z}_q[x]/(x^{n/2} - 1)$$

$$x \mapsto \omega_n x$$

Thus, as for $\mathbb{Z}_q[x]/(x^n - 1)$, there is

$$\Psi\Phi : \mathbb{Z}_q[x]/(x^n - 1) \;\cong\; \mathbb{Z}_q[x]/(x^{n/2} - 1) \times \mathbb{Z}_q[x]/(x^{n/2} - 1)$$

$$\boldsymbol{a} \mapsto (\boldsymbol{a}', \boldsymbol{a}'')$$

and the detailed functioning process

$$(\Psi\Phi)\left(\sum_{i=0}^{n-1} a_i x^i\right) = \left(\sum_{i=0}^{n/2-1} (a_i + a_{i+n/2})x^i, \sum_{i=0}^{n/2-1} \omega_n^i \cdot (a_i - a_{i+n/2})x^i\right)$$

$$(\Psi\Phi)^{-1}\left(\sum_{i=0}^{n/2-1} a'_i x^i, \sum_{i=0}^{n/2-1} a''_i x^i\right) = \sum_{i=0}^{n/2-1} \frac{a'_i + \omega_n^{-i} \cdot a''_i}{2} x^i + \sum_{i=0}^{n/2-1} \frac{a'_i - \omega_n^{-i} \cdot a''_i}{2} x^{i+n/2}.$$

As for the forward twisted FFT trick, for example, $\boldsymbol{a} \in \mathbb{Z}_q[x]/(x^n - 1)$ in the 0-th level generates $\boldsymbol{a}'$ and $\boldsymbol{a}''$, where

$$a'_i = a_i + a_{i+\frac{n}{2}}, \;\; a''_i = \omega_n^i \cdot (a_i - a_{i+\frac{n}{2}}) \qquad (i = 0, 1, \ldots, n/2 - 1),$$

where Gentleman-Sande algorithm are used.

It can be applied repeatedly to map $\mathbb{Z}_q[x]/(x^{n/2} - 1)$, and down to linear terms such as $\mathbb{Z}_q[x]/(x \mp 1)$.

Specifically, in the k-th level, the similar isomorphism $\Psi : x \mapsto \omega_n^{2^{k-1}} x$ is applied from $\mathbb{Z}_q[x]/(x^{n/2^k} + 1)$ to $\mathbb{Z}_q[x]/(x^{n/2^k} - 1)$, $1 \leq k < \log n$.

The complete process of twisted FFT trick on mapping $\mathbb{Z}_q[x]/(x^n - 1)$ is shown in Figure 4.

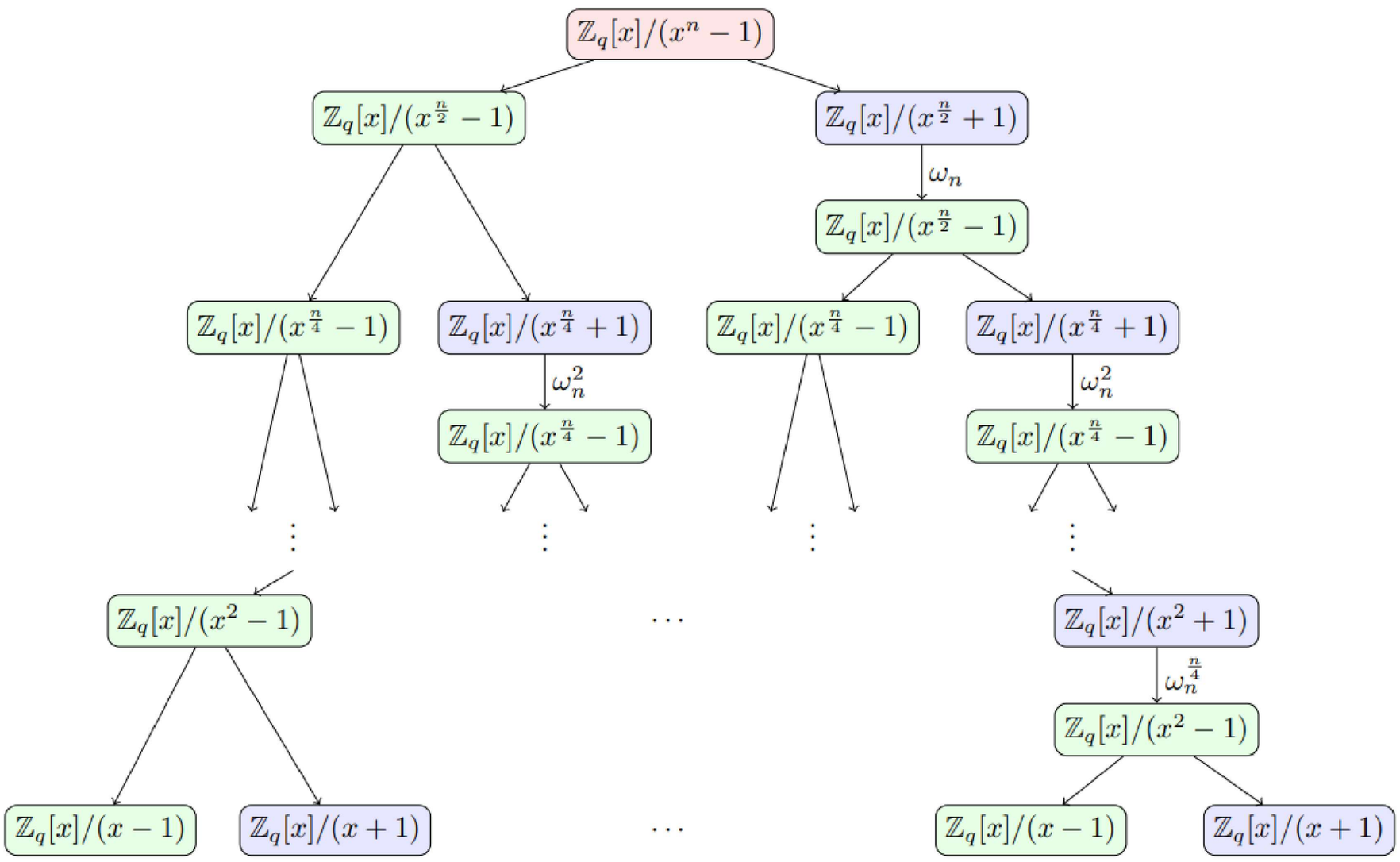
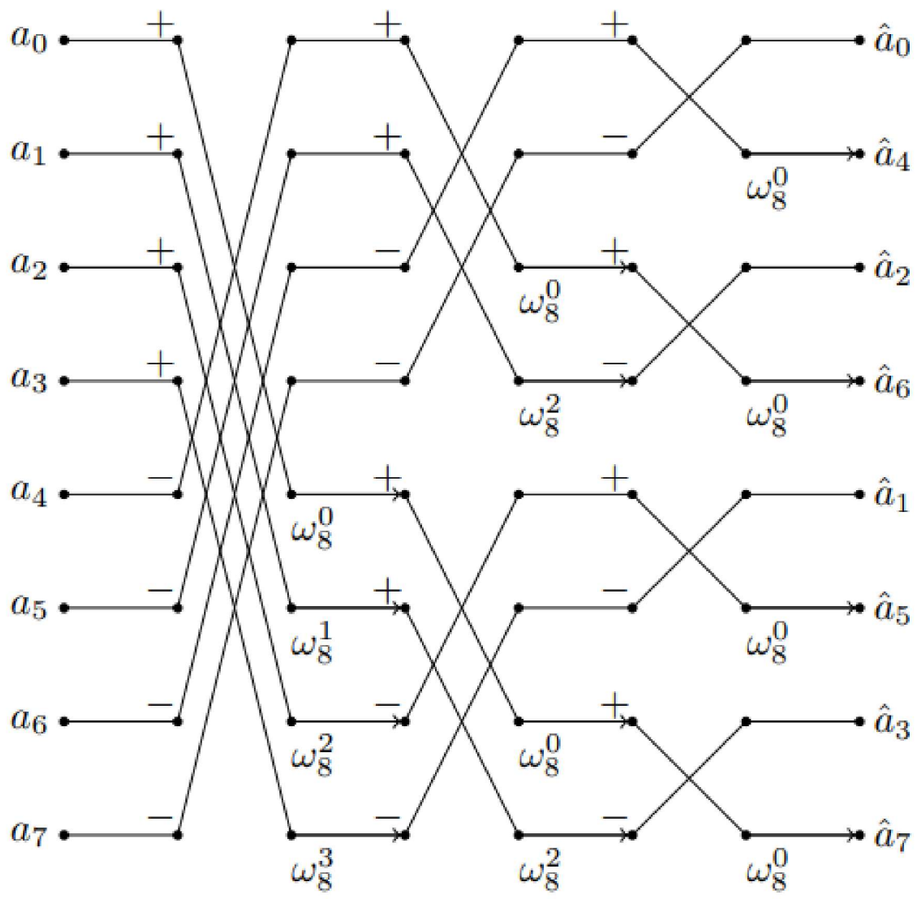


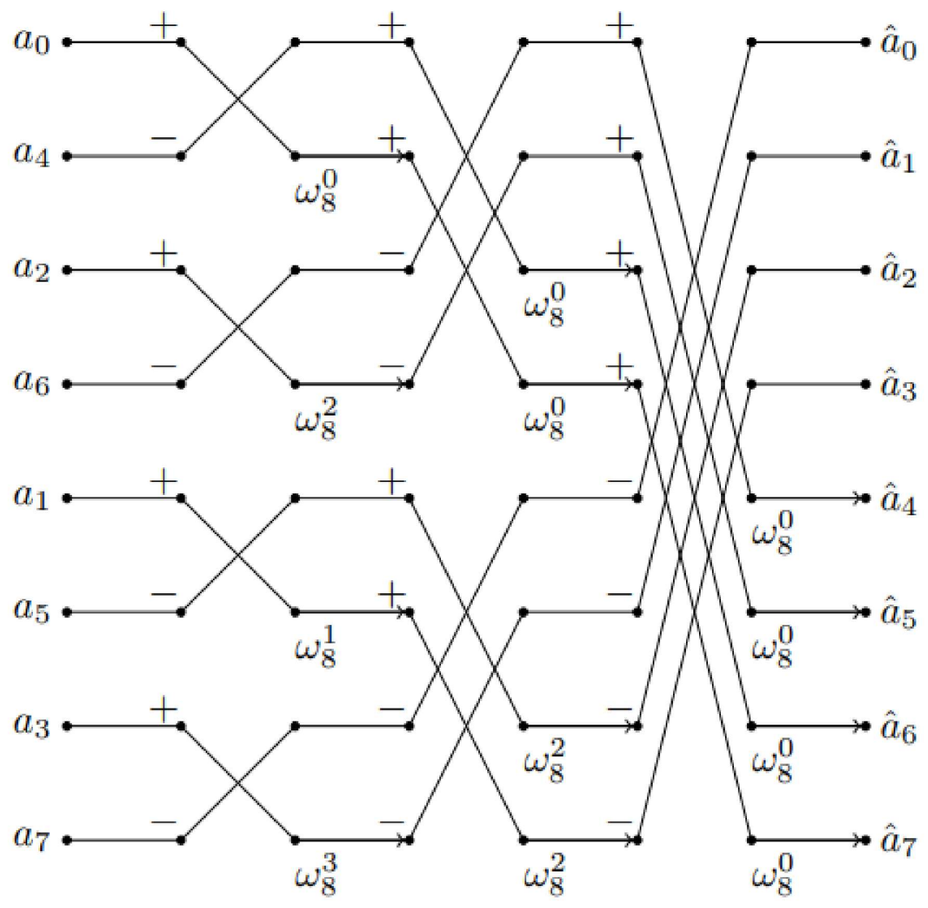
Fig. 4. CRT map of twisted FFT trick over $\mathbb{Z}_q[x]/(x^n - 1)$

Such GS NTT algorithm is denoted by $\text{NTT}^{GS}_{no \to bo}$.



(d) $\mathrm{NTT}^{GS}_{no\to bo}$

Adjust the input/output order, and we obtain $\mathrm{NTT}^{GS}_{bo\to no}$.

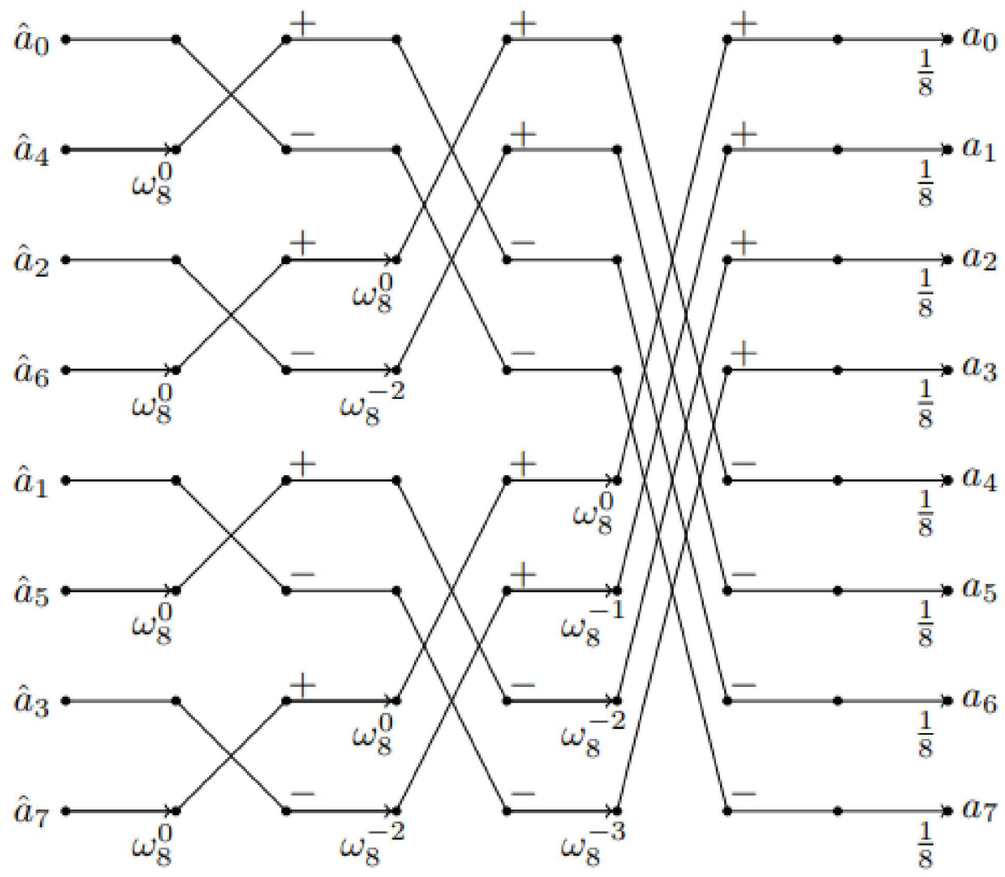


(b) $\mathrm{NTT}^{GS}_{bo\to no}$

The inverse twisted FFT trick is computed in much the same way by iteratively inverting the above process. For example, the process of computing $\boldsymbol{a}$ in the 0-th level from $\boldsymbol{a}'$ and $\boldsymbol{a}''$ in the first level with Cooley-Tukey butterflies is as follows

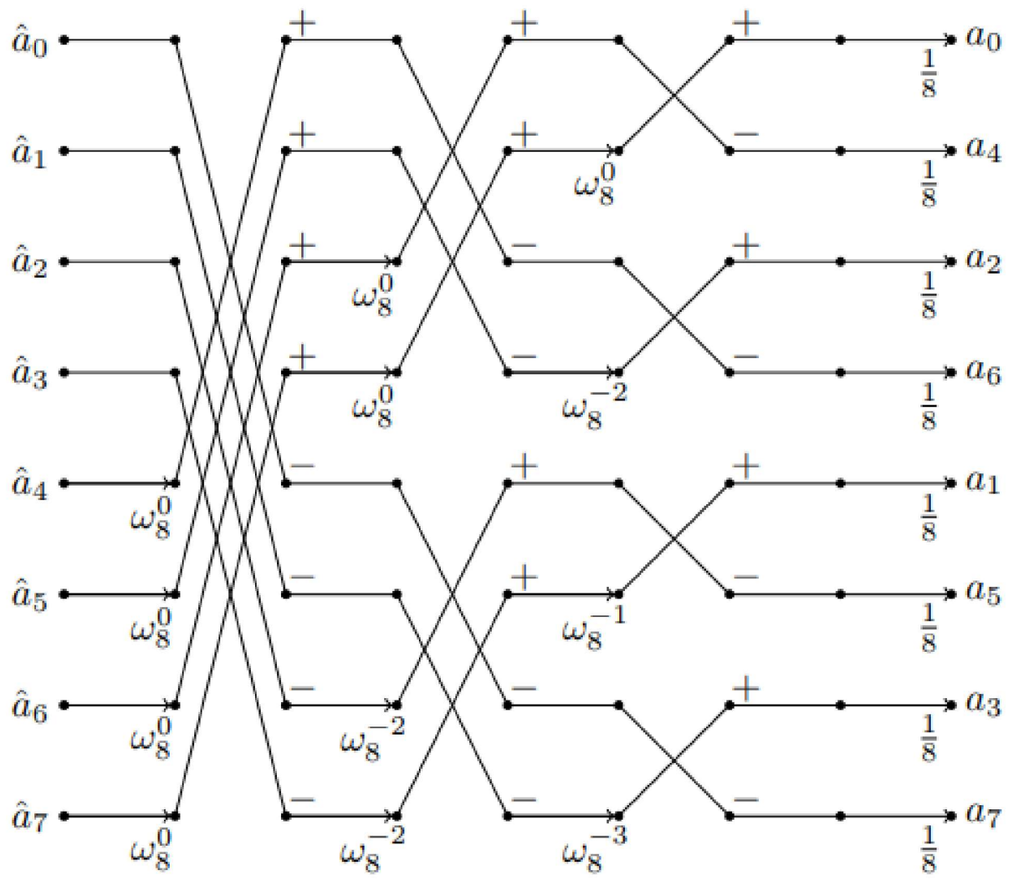
$$a_i = (a'_i + \omega_n^{-i} \cdot a''_i)/2, \;\; a_{i+n/2} = (a'_i - \omega_n^{-i} \cdot a''_i)/2 \qquad (i = 0, 1, \ldots, n/2 - 1).$$

Such computing from the $(k+1)$-th level to the k-th level can be achieved in the same way, where $1 \leq k < \log n$. The scale factor 2 in each level can be omitted, with multiplying the final result by n^{-1} in the end. We denote this type of CT INTT by $\mathrm{INTT}^{CT}_{bo\to no}$.



(a) $\text{INTT}^{CT}_{bo\rightarrow no}$

Adjust its input/output order and the new transform is denoted by $\text{INTT}^{CT}_{no\rightarrow bo}$.



(c) $\text{INTT}^{CT}_{no\rightarrow bo}$

Notice that there only exists fast algorithm based on Cooley-Tukey butterfly for NTT^{ψ}, and that based on Gentleman-Sande butterfly for $\text{INTT}^{\psi^{-1}}$. This is because, once we use Gentleman-Sande butterfly to compute NTT^{ψ} or use Cooley-Tukey butterfly to compute $\text{INTT}^{\psi^{-1}}$, the ψ_{2n} term can not be further processed.

3.5 RADIX-2 FAST NUMBER THEORETIC TRANSFORM FROM FFT PERSPECTIVES

This section describes NTT algorithms from FFT perspectives.

The basic principle of fast NTT algorithms is using "divide and conquer" skill to divide the n-point NTT into two $n/2$-point NTTs, based on the periodicity and symmetry of the primitive root of unity.

The properties of primitive roots of unity in NTT are similar to those of twiddle factors in FFT.

The primitive n-th root of unity ω_n in $\mathbb{Z}_q$ has the following properties

$$\text{periodicity: } \omega_n^{k+n} = \omega_n^k$$

$$\text{symmetry: } \omega_n^{k+n/2} = -\omega_n^k$$

where k is a non-negative integer.

It is trivial that the primitive $2n$-th root of unity ψ_{2n} shares similar properties if ψ_{2n} exists.

Cooley-Tukey Algorithm for CC-based NTT

1. NTT

   Based on the parity of the indexes of the coefficients a_i in $\boldsymbol{a}$, the terms can be separated into two parts, for $j = 0, 1, \ldots, n-1$,

$$\begin{aligned}\hat{a}_j &= \sum_{i=0}^{n/2-1} a_{2i}\omega_n^{2ij} + \sum_{i=0}^{n/2-1} a_{2i+1}\omega_n^{(2i+1)j} \bmod q \\ &= \sum_{i=0}^{n/2-1} a_{2i}(\omega_n^2)^{ij} + \omega_n^j \sum_{i=0}^{n/2-1} a_{2i+1}(\omega_n^2)^{ij} \bmod q.\end{aligned}$$

   Based on the periodicity and symmetry of the primitive root of unity, for $j = 0, 1, \ldots, n/2-1$,

$$\hat{a}_j = \sum_{i=0}^{n/2-1} a_{2i}(\omega_n^2)^{ij} + \omega_n^j \sum_{i=0}^{n/2-1} a_{2i+1}(\omega_n^2)^{ij} \bmod q$$

$$\hat{a}_{j+n/2} = \sum_{i=0}^{n/2-1} a_{2i}(\omega_n^2)^{ij} - \omega_n^j \sum_{i=0}^{n/2-1} a_{2i+1}(\omega_n^2)^{ij} \bmod q.$$

   Let

$$\hat{a}'_j = \sum_{i=0}^{n/2-1} a_{2i}(\omega_n^2)^{ij} \bmod q,$$

$$\hat{a}''_j = \sum_{i=0}^{n/2-1} a_{2i+1}(\omega_n^2)^{ij} \bmod q, \quad j = 0, 1, \ldots, n/2-1.$$

   Formula can be rewritten as

$$\hat{a}_j = \hat{a}'_j + \omega_n^j \hat{a}''_j \bmod q,$$

$$\hat{a}_{j+n/2} = \hat{a}'_j - \omega_n^j \hat{a}''_j \bmod q, \quad j = 0, 1, \ldots, n/2-1.$$

   One can learn from the definition of NTT that, $\hat{a}'_j$ and $\hat{a}''_j$ can be computed by $n/2$-point NTT, from the even-indexed and the odd-indexed coefficients of $\boldsymbol{a}$, respectively.

   Formula shows that, the original n-point NTT can be divided into two $n/2$-point NTTs by "divide-and-conquer" method.

   After getting the $n/2$-point NTT results $\hat{a}'_j$ and $\hat{a}''_j$, the original n-point $\hat{\boldsymbol{a}}$ can be easily achieved by multiplying ω_n^j and simple additions/subtractions.

   This kind of "divide and conquer" skill can also be applied to compute $\hat{a}'_j$ and $\hat{a}''_j$.

Since n is a power of two, it can be separated down to 2-point NTTs.

Note that, the coefficients of the input polynomials are indexed under bit-reversed order, while the coefficients of the output polynomials are indexed under natural order.

2. INTT

Cooley-Tukey butterfly can also be applied to compute INTT. In contrast to NTT, there is extra multiplications by a scale factor n^{-1} in INTT.

With neglecting n^{-1}, the terms in the summation are the same as those in NTT formula, except replacing ω_n with ω_n^{-1}.

For the convenience of understanding, its brief computing process is given here.

$$a_i = \sum_{j=0}^{n/2-1} \hat{a}_{2j}(\omega_n^2)^{-ij} + \omega_n^{-i} \sum_{j=0}^{n/2-1} \hat{a}_{2j+1}(\omega_n^2)^{-ij} \bmod q. \quad i = 0, 1, \ldots, n-1$$

Let

$$a'_i = \sum_{j=0}^{n/2-1} \hat{a}_{2j}(\omega_n^2)^{-ij} \bmod q,$$

$$a''_i = \sum_{j=0}^{n/2-1} \hat{a}_{2j+1}(\omega_n^2)^{-ij} \bmod q, \quad i = 0, 1, \ldots, n/2-1.$$

$$a_i = a'_i + \omega_n^{-i} a''_i \bmod q$$

$$a_{i+n/2} = a'_i - \omega_n^{-i} a''_i \bmod q, \quad i = 0, 1, \ldots, n/2-1.$$

Therefore, the computing of n-point INTT can be divided into two $n/2$-point INTTs, which can be done down to 2-point INTTs finally.

Gentlemen-Sande Algorithm for CC-based NTT

1. NTT

Gentlemen-Sande algorithm separates the coefficients of $\boldsymbol{a}$ into the upper half and the lower half. Specifically, for $j = 0, 1, \ldots, n-1$,

$$\hat{a}_j = \sum_{i=0}^{n/2-1} a_i \omega_n^{ij} + \sum_{i=n/2}^{n-1} a_i \omega_n^{ij} \bmod q$$

$$= \sum_{i=0}^{n/2-1} a_i \omega_n^{ij} + \sum_{i=0}^{n/2-1} a_{i+n/2} \omega_n^{(i+n/2)j} \bmod q.$$

Based on the periodicity and symmetry of the primitive root of unity, the terms $\hat{a}_j$ continue to be dealt

with according to the parity of index j. For $j = 0, 1, \ldots, n/2 - 1$,

$$\hat{a}_{2j} = \sum_{i=0}^{n/2-1} a_i \omega_n^{2ij} + (-1)^{2j} \cdot \sum_{i=0}^{n/2-1} a_{i+n/2} \omega_n^{2ij} \bmod q$$

$$= \sum_{i=0}^{n/2-1} (a_i + a_{i+n/2})(\omega_n^2)^{ij} \bmod q,$$

$$\hat{a}_{2j+1} = \sum_{i=0}^{n/2-1} a_i \omega_n^{(2j+1)i} + (-1)^{2j+1} \cdot \sum_{i=0}^{n/2-1} a_{i+n/2} \omega_n^{(2j+1)i} \bmod q$$

$$= \sum_{i=0}^{n/2-1} \left[(a_i - a_{i+n/2}) \cdot \omega_n^i\right] (\omega_n^2)^{ij} \bmod q.$$

Let

$$b'_i = a_i + a_{i+n/2} \bmod q,$$

$$b''_i = (a_i - a_{i+n/2}) \cdot \omega_n^i \bmod q, i = 0, 1, \ldots, n/2 - 1.$$

The above formula can be rewritten as

$$\hat{a}_{2j} = \sum_{i=0}^{n/2-1} b'_i (\omega_n^2)^{ij} \bmod q,$$

$$\hat{a}_{2j+1} = \sum_{i=0}^{n/2-1} b''_i (\omega_n^2)^{ij} \bmod q, \quad j = 0, 1, \ldots, n/2 - 1.$$

One can learn from the definition of NTT that, formula is exact the $n/2$-point NTTs with respect to b'_i, b''_i $(i = 0, 1, \ldots, n/2 - 1)$.

Thus, after deriving b'_i, b''_i from a_i, the original n-point NTT is transformed into $n/2$-point NTTs with respect to b'_i, b''_i.

Similarly, the $n/2$-point NTT can be transformed into $n/4$-point NTTs, and down to 2-point NTTs.

2. INTT

Gentleman-Sande butterfly can be similarly applied to compute INTT, by neglecting n^{-1} and replacing ω_n with ω_n^{-1} in NTT. That is, for $i = 0, 1, \ldots, n/2 - 1$,

$$a_{2i} = \sum_{j=0}^{n/2-1} (\hat{a}_j + \hat{a}_{j+n/2})(\omega_n^2)^{-ij} \bmod q$$

$$a_{2i+1} = \sum_{i=0}^{n/2-1} \left[(\hat{a}_j - \hat{a}_{j+n/2}) \cdot \omega_n^{-j}\right] (\omega_n^2)^{-ij} \bmod q.$$

Let

$$\hat{b}'_j = \hat{a}_j + \hat{a}_{j+n/2} \bmod q,$$

$$\hat{b}''_j = (\hat{a}_j - \hat{a}_{j+n/2}) \cdot \omega_n^{-j} \bmod q, \quad j = 0, 1, \ldots, n/2 - 1.$$

Formula can be rewritten as:

$$a_{2i} = \sum_{j=0}^{n/2-1} \hat{b}'_j (\omega_n^2)^{-ij} \bmod q$$

$$a_{2i+1} = \sum_{j=0}^{n/2-1} \hat{b}''_j (\omega_n^2)^{-ij} \bmod q, \quad i = 0, 1, \ldots, n/2-1.$$

Similarly, n-point INTT can be divided into two $n/2$-point INTTs, and down to 2-point INTTs.

Radix-2 Fast NWC-based NTT

The process of multiplying the coefficients by ψ_{2n}^i before forward transform is referred to as pre-processing,

while the process of multiplying the coefficients by ψ_{2n}^{-i} after inverse transform is referred to as post-processing.

Fast algorithms for NWC-based NTT, such as NTT^{ψ} and $\text{INTT}^{\psi^{-1}}$, can be constructed by using radix-2 CT/GS NTT/INTT algorithm with pre-processing and post-processing.

However, such construction requires extra point-wise multiplication with $\boldsymbol{\psi}/\boldsymbol{\psi}^{-1}$ besides radix-2 algorithms, resulting with n extra multiplications. In fact, these additional multiplications are not necessary.

- Roy et al. integrate the pre-processing about $\boldsymbol{\psi}$ into $\text{NTT}^{CT}_{bo\to no}$.
- Pöppelmann et al. integrate the post-processing about $\boldsymbol{\psi}^{-1}$ into $\text{INTT}^{GS}_{bo\to no}$.
- Furthermore, Zhang et al. integrate $\boldsymbol{\psi}^{-1}$ and n^{-1} into $\text{INTT}^{GS}_{bo\to no}$.

1. Cooley-Tukey algorithm for NTT^{ψ}

   According to the definition of NTT^{ψ}, the coefficients of $\hat{\boldsymbol{a}}$ can be written as

   $$\hat{a}_j = \sum_{i=0}^{n-1} a_i \psi_{2n}^i \omega_n^{ij} \bmod q, \quad j = 0, 1, \ldots, n-1.$$

   Divide the summation into two parts based on the parity of the index of a_i, and for $j = 0, 1, \ldots, n-1$, we get

   $$\hat{a}_j = \sum_{i=0}^{n/2-1} a_{2i} \omega_n^{2ij} \psi_{2n}^{2i} + \sum_{i=0}^{n/2-1} a_{2i+1} \omega_n^{(2i+1)j} \psi_{2n}^{2i+1} \bmod q$$

   $$= \sum_{i=0}^{n/2-1} a_{2i} (\omega_n^2)^{ij} (\psi_{2n}^2)^i + \omega_n^j \psi_{2n} \sum_{i=0}^{n/2-1} a_{2i+1} (\omega_n^2)^{ij} (\psi_{2n}^2)^i \bmod q.$$

   Based on the periodicity and symmetry of the primitive root of unity, for $j = 0, 1, \ldots, n/2-1$, we get

   $$\hat{a}_j = \sum_{i=0}^{n/2-1} a_{2i} (\omega_n^2)^{ij} (\psi_{2n}^2)^i + \omega_n^j \psi_{2n} \sum_{i=0}^{n/2-1} a_{2i+1} (\omega_n^2)^{ij} (\psi_{2n}^2)^i \bmod q$$

   $$\hat{a}_{j+n/2} = \sum_{i=0}^{n/2-1} a_{2i} (\omega_n^2)^{ij} (\psi_{2n}^2)^i - \omega_n^j \psi_{2n} \sum_{i=0}^{n/2-1} a_{2i+1} (\omega_n^2)^{ij} (\psi_{2n}^2)^i \bmod q.$$

Let

$$\hat{a}'_j = \sum_{i=0}^{n/2-1} a_{2i}(\omega_n^2)^{ij}(\psi_{2n}^2)^i \bmod q,$$

$$\hat{a}''_j = \sum_{i=0}^{n/2-1} a_{2i+1}(\omega_n^2)^{ij}(\psi_{2n}^2)^i \bmod q, \quad j = 0, 1, \ldots, n/2-1.$$

With $\omega_n^j \psi_{2n} = \psi_{2n}^{2j+1}$, the above formula can be rewritten as, for $j = 0, 1, \ldots, n/2-1$:

$$\hat{a}_j = \hat{a}'_j + \psi_{2n}^{2j+1}\hat{a}''_j \bmod q, \quad \hat{a}_{j+n/2} = \hat{a}'_j - \psi_{2n}^{2j+1}\hat{a}''_j \bmod q.$$

One can see that, $\hat{a}'_j$ and $\hat{a}''_j$ can be obtained via exact $n/2$-point NTT$^\psi$s.

2. Gentleman-Sande algorithm for INTT$^{\psi^{-1}}$

   According to the definition of INTT$^{\psi^{-1}}$, the coefficients of $\boldsymbol{a}$ can be written as

$$a_i = n^{-1}\psi_{2n}^{-i}\sum_{j=0}^{n-1}\hat{a}_j\omega_n^{-ij} \bmod q, \quad i = 0, 1, \ldots, n-1.$$

   With neglecting n^{-1}, the summation can be divided into the upper half and the lower half with respect to the index of $\hat{a}_j$.

   For $i = 0, 1, \ldots, n-1$,

$$a_i = \psi_{2n}^{-i}\left(\sum_{j=0}^{n/2-1}\hat{a}_j\omega_n^{-ij} + \sum_{j=n/2}^{n-1}\hat{a}_j\omega_n^{-ij}\right)$$

$$= \psi_{2n}^{-i}\left[\sum_{j=0}^{n/2-1}\hat{a}_j\omega_n^{-ij} + \sum_{j=0}^{n/2-1}\hat{a}_{j+n/2}\omega_n^{-i(j+n/2)}\right] \bmod q.$$

Based on the periodicity and symmetry of the primitive root of unity, for $i = 0, 1, \ldots, n/2-1$,

$$a_{2i} = \psi_{2n}^{-2i}\left[\sum_{j=0}^{n/2-1}\hat{a}_j\omega_n^{-2ij} + (-1)^{2i}\cdot\sum_{j=0}^{n/2-1}\hat{a}_{j+n/2}\omega_n^{-2ij}\right] \bmod q$$

$$= (\psi_{2n}^2)^{-i}\sum_{j=0}^{n/2-1}(\hat{a}_j + \hat{a}_{j+n/2})(\omega_n^2)^{-ij} \bmod q,$$

$$a_{2i+1} = \psi_{2n}^{-(2i+1)}\left[\sum_{j=0}^{n/2-1}\hat{a}_j\omega_n^{-(2i+1)j} + (-1)^{2i+1}\cdot\sum_{j=0}^{n/2-1}\hat{a}_{j+n/2}\omega_n^{-(2i+1)j}\right] \bmod q$$

$$= (\psi_{2n}^2)^{-i}\sum_{j=0}^{n/2-1}\left[(\hat{a}_j - \hat{a}_{j+n/2})\cdot\omega_n^{-j}\psi_{2n}^{-1}\right](\omega_n^2)^{-ij} \bmod q.$$

Since $\omega_n^{-j}\psi_{2n}^{-1} = \psi_{2n}^{-(2j+1)}$, letting

$$\hat{b}'_j = \hat{a}_j + \hat{a}_{j+n/2} \bmod q,$$

$$\hat{b}''_j = (\hat{a}_j - \hat{a}_{j+n/2})\cdot\psi_{2n}^{-(2j+1)} \bmod q, \quad j = 0, 1, \ldots, n/2-1,$$

the above formula can be rewritten as, for $i = 0, 1, \ldots, n/2 - 1$,

$$a_{2i} = (\psi_{2n}^2)^{-i} \sum_{j=0}^{n/2-1} \hat{b}'_j (\omega_n^2)^{-ij} \bmod q,$$

$$a_{2i+1} = (\psi_{2n}^2)^{-i} \sum_{j=0}^{n/2-1} \hat{b}''_j (\omega_n^2)^{-ij} \bmod q.$$

Computing n-point INTT$^{\psi^{-1}}$ can be transformed into two $n/2$-point INTT$^{\psi^{-1}}$s with respect to $\hat{b}'_j, \hat{b}''_j$. Forthermore, $\boldsymbol{\psi}^{-1}$ and n^{-1} can both be integrated into INTT$^{GS}_{bo \to no}$.

Thus, n^{-1} is no longer neglected, and for $i = 0, 1, \ldots, n/2 - 1$,

$$a_{2i} = \left(\frac{n}{2}\right)^{-1} (\psi_{2n}^2)^{-i} \sum_{j=0}^{n/2-1} \hat{b}'_j (\omega_n^2)^{-ij} \bmod q,$$

$$a_{2i+1} = \left(\frac{n}{2}\right)^{-1} (\psi_{2n}^2)^{-i} \sum_{j=0}^{n/2-1} \hat{b}''_j (\omega_n^2)^{-ij} \bmod q,$$

$$\text{where} \quad \hat{b}'_j = (\hat{a}_j + \hat{a}_{j+n/2})/2 \mod q,$$

$$\hat{b}''_j = (\hat{a}_j - \hat{a}_{j+n/2})/2 \cdot \psi_{2n}^{-(2j+1)} \bmod q, \quad j = 0, 1, \ldots, n/2 - 1.$$

When computing $\hat{b}'_j$ and $\hat{b}''_j$, the scale factor 2 will be dealt with directly, by using addition and displacement (i.e., "$\gg$") to compute $x/2 \bmod q$.

$$x/2 \equiv \begin{cases} (x \gg 1) \bmod q, & x \text{ is even}, \\ (x \gg 1) + (q+1)/2 \bmod q, & x \text{ is odd}. \end{cases}$$

3.6 In-place Operation, Reordering and Complexity

In-place operation

The Cooley-Tukey butterfly and Gentleman-Sande butterfly store the input and output data in the same address before and after the computing process, i.e., read data from some storage address for the computation, where the computing results are stored. This kind of operation is referred to as in-place operation. Obviously, there is no need of extra storage for in-place operation.

Reordering

The input and output order of the polynomial coefficients have to be taken into consideration.

Although the coefficients of the practical input polynomial are under natural order,

the output after NTT$^{CT}_{no \to bo}$/NTT$^{GS}_{no \to bo}$ will end under bit-reversed order,

which is the required order for input of INTT$^{CT}_{bo \to no}$/INTT$^{GS}_{bo \to no}$,

but not the required one for input of INTT$^{CT}_{no \to bo}$/INTT$^{GS}_{no \to bo}$.

In this case, extra reordering is needed from bit-reversed order to natural order.

Besides, if NTT is conducted via $\mathrm{NTT}^{CT}_{bo\to no}$ and $\mathrm{NTT}^{GS}_{bo\to no}$, the input polynomial is supposed to be reordered from natural order to bit-reversed order.

Similarly, there is a requirement on reordering the output polynomial of $\mathrm{INTT}^{CT}_{no\to bo}$ and $\mathrm{INTT}^{GS}_{no\to no}$.

In a word, there is a way for cyclic convolution-based polynomial multiplication without reordering, i.e., for $\ddagger, \natural \in \{CT, GS\}$

$$\boldsymbol{c} = \mathrm{INTT}^{\ddagger}_{bo\to no}\left(\mathrm{NTT}^{\natural}_{no\to bo}(\boldsymbol{a}) \circ \mathrm{NTT}^{\natural}_{no\to bo}(\boldsymbol{b})\right).$$

Similarly, there is a way for NWC-based polynomial multiplication without extra reordering:

$$\boldsymbol{c} = \mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}\left(\mathrm{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{a}) \circ \mathrm{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{b})\right).$$

Complexity

The complexities of NTT/INTT are given in Table.

Each Cooley-Tukey butterfly consumes one multiplication and two additions (subtractions), where ωb is computed once and can be used twice.

Similar analysis can also be applied to Gentleman-Sande butterfly.

All the fast algorithms consist of $\log n$ levels, where there are $n/2$ butterfly operations on each level. As for the inverse transforms, their complexities require extra n multiplications because of dealing with the scale factor n^{-1}.

All the complexity of the polynomial multiplication based on these NTT fast algorithms is $O(n \log n)$, which has a significant advantage over that of polynomial multiplication based on directly-computing NTT/INTT, or any other polynomial multiplication algorithms such as the schoolbook algorithm and Karatsuba/Toom-Cook algorithm.

NTT algorithms	Multiplication complexities
$\mathrm{NTT}, \mathrm{INTT}, \mathrm{NTT}^{\psi}, \mathrm{INTT}^{\psi^{-1}}$	$O(n^2)$
$\mathrm{NTT}^{\natural}_{no\to bo}, \mathrm{NTT}^{\natural}_{bo\to no}, \mathrm{NTT}^{CT,\psi}_{no\to bo}, \mathrm{NTT}^{CT,\psi}_{bo\to no}$	$\frac{1}{2}n\log n$
$\mathrm{INTT}^{\ddagger}_{no\to bo}, \mathrm{INTT}^{\ddagger}_{bo\to no}, \mathrm{INTT}^{GS,\psi^{-1}}_{no\to bo}, \mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}$	$\frac{1}{2}n\log n + n$
$\mathrm{NTT}^{\natural}_{no\to bo} \circ \psi, \mathrm{NTT}^{\natural}_{bo\to no} \circ \psi_{bo}$	$\frac{1}{2}n\log n + n$
$\psi^{-1}_{bo} \circ \mathrm{INTT}^{\ddagger}_{no\to bo}, \psi^{-1} \circ \mathrm{INTT}^{\ddagger}_{bo\to no}$	$\frac{1}{2}n\log n + 2n$

$\ddagger, \natural \in \{CT, GS\}$. ψ_{bo} and ψ^{-1}_{bo} mean that the coefficients are under bit-reversed order. n is the length of NTT.

4 METHODS TO WEAKEN RESTRICTIONS ON PARAMETER CONDITIONS OF NTT

The full CC-based NTT requires that the parameter n is a power of two and q is prime satisfying $q \equiv 1 \pmod{n}$, while the full NWC-based NTT requires that the parameter n is a power of two and q is prime satisfying $q \equiv 1 \pmod{2n}$

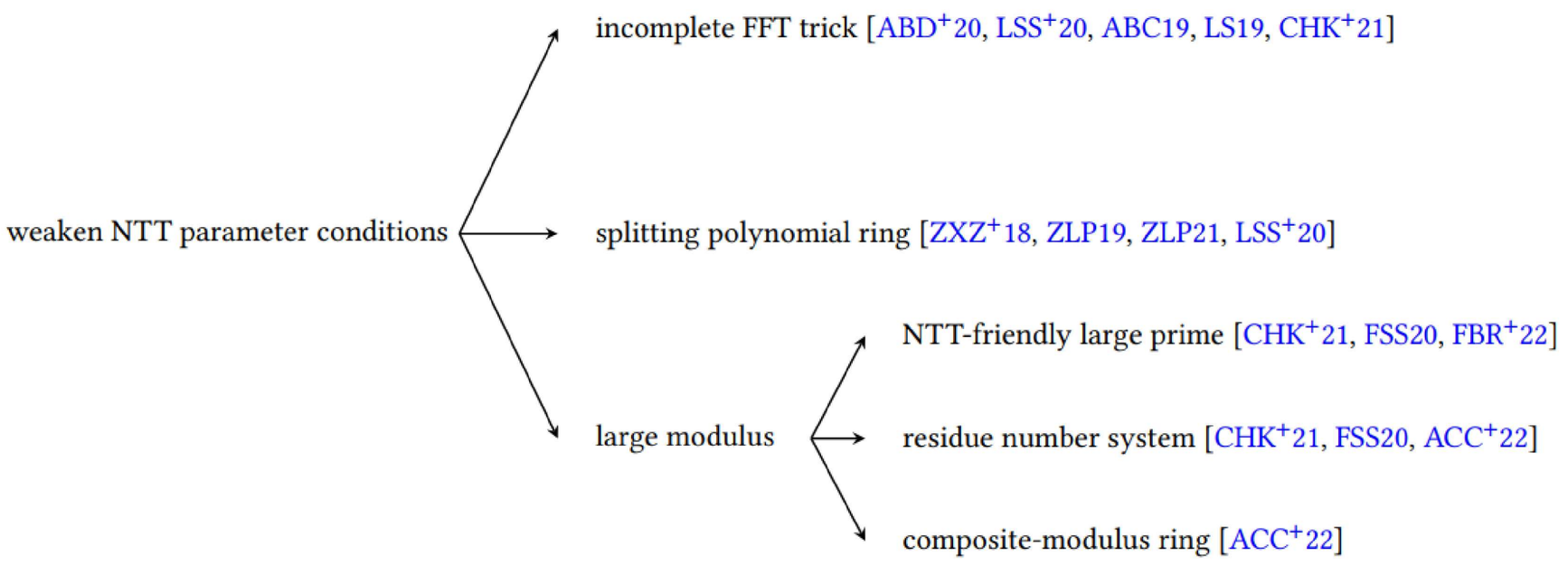
Traditionally, NTT puts some restrictions on its parameters. In recent years, many research efforts are made for NTT's restrictions on parameters and a series of methods have been proposed to weaken them.

This section introduces the recent advances of weakening parameter restrictions with respect to $\mathbb{Z}_q[x]/(x^n \pm 1)$ where n is a power of two. Those methods can be mainly classified into the following three categories.

- Method based on incomplete FFT trick
- Method based on splitting polynomial ring
- Method based on large modulus

The first two methods are applied for the case that the modulus q is an NTT-friendly prime of the form $q = q' \cdot 2^e + 1$ but q can not lead to a full NTT.

The last method is applied for the case that the modulus q is an NTT-unfriendly prime.



4.1 Method Based on Incomplete FFT trick

Introduce the method based on the incomplete FFT trick over $\mathbb{Z}_q[x]/(x^n + 1)$ where n is a power of two and q is NTT-friendly prime but does not satisfy $q \equiv 1 \pmod{2n}$.

Then extend the ring to $\mathbb{Z}_q[x]/(x^n - 1)$ where n is a power of two and q is NTT-friendly prime but does not satisfy $q \equiv 1 \pmod{n}$.

Method based on incomplete FFT trick over $\mathbb{Z}_q[x]/(x^n + 1)$

Fully-mapping FFT trick means to map $\mathbb{Z}_q[x]/(x^n + 1)$ down to linear terms, e.g., $\mathbb{Z}_q[x]/(x - \psi_{2n}^{2i+1})$. The condition $q \equiv 1 \pmod{2n}$ is required such that the primitive $2n$-th root of unity ψ_{2n} exits.

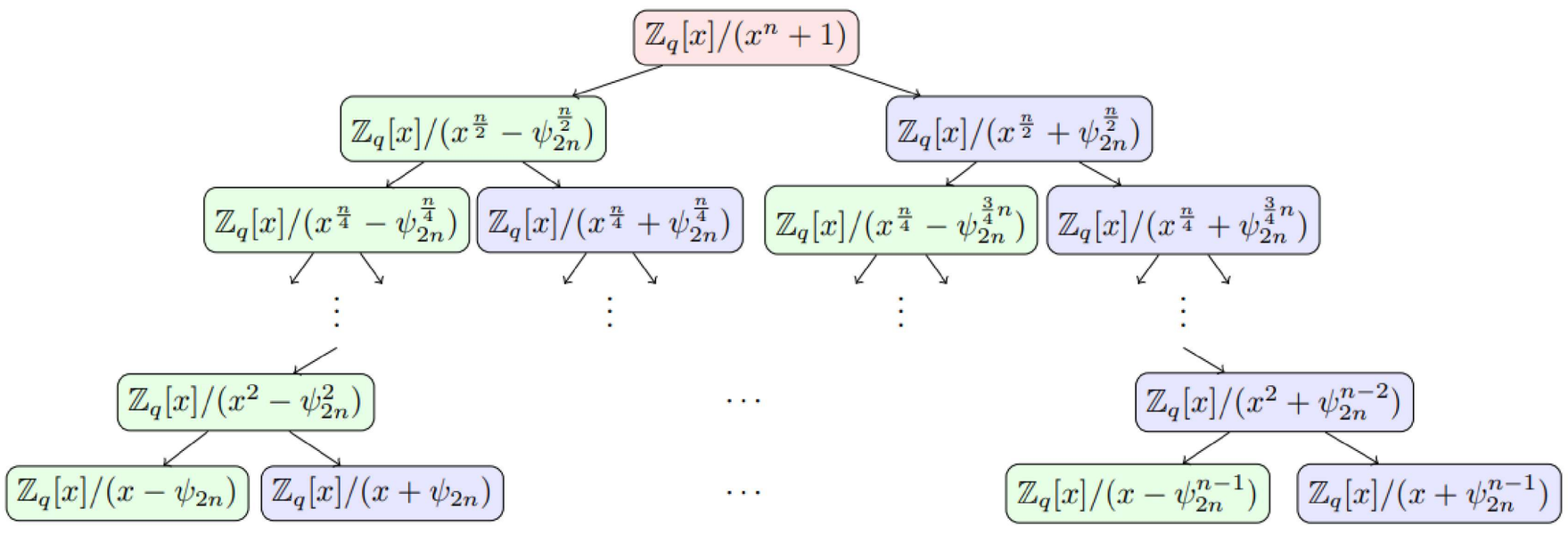


Fig. 3. CRT map of FFT trick over $\mathbb{Z}_q[x]/(x^n + 1)$

Moenck noticed that FFT trick does not have to map down to linear terms, and one can stop its mapping before the last β level, $\beta = 0, 1, \ldots, \log n - 1$.

In detail, its CRT map of $\mathbb{Z}_q[x]/(x^n + 1)$ is as follows.

$$\mathbb{Z}_q[x]/(x^n + 1) \cong \prod_{i=0}^{n/2^\beta - 1} \mathbb{Z}_q[x]/(x^{2^\beta} - \psi_{2n/2^\beta}^{2\mathrm{brv}_{n/2^\beta}(i)+1}).$$

It is referred to as "Incomplete NTT" or "Truncated-NTT".

The reason is that its CRT tree map is obtained by cropping the last β levels from fully-mapping $(\log n)$-level FFT trick tree map.

Note that after forward transforms, $\boldsymbol{a}$'s images in $\mathbb{Z}_q[x]/(x^{2^\beta} - \psi_{2n/2^\beta}^{2\mathrm{brv}_{n/2^\beta}(i)+1})$ are degree-$(2^\beta - 1)$ polynomials.

The point-wise multiplication is performed about the corresponding degree-$(2^\beta - 1)$ polynomials in $\mathbb{Z}_q[x]/(x^{2^\beta} - \psi_{2n/2^\beta}^{2\mathrm{brv}_{n/2^\beta}(i)+1})$.

As for the inverse transforms, the scale factor 2 is omitted in every level, followed by multiplying by a total scalar $(n/2^\beta)^{-1}$ in the end. The forward/inverse transforms with β levels cropped are denoted by $\mathrm{NTT}_{no\to bo,\beta}^{CT,\psi}/\mathrm{INTT}_{bo\to no,\beta}^{GS,\psi^{-1}}$ respectively, where $\beta = 0, 1, \ldots, \log n - 1$.

Obviously, they are exactly $\mathrm{NTT}_{no\to bo}^{CT,\psi}$ and $\mathrm{INTT}_{bo\to no}^{GS,\psi^{-1}}$ if $\beta = 0$.

The restriction on n and q can be weakened to $q \equiv 1 \pmod{2n/2^\beta}$.

The way to compute polynomial multiplication is the general form of formula

$$\boldsymbol{c} = \mathrm{INTT}_{bo\to no}^{GS,\psi^{-1}}\left(\mathrm{NTT}_{no\to bo}^{CT,\psi}(\boldsymbol{a}) \circ \mathrm{NTT}_{no\to bo}^{CT,\psi}(\boldsymbol{b})\right),$$

that is

$$\boldsymbol{c} = \mathrm{INTT}_{bo\to no,\beta}^{GS,\psi^{-1}}\left(\mathrm{NTT}_{no\to bo,\beta}^{CT,\psi}(\boldsymbol{a}) \circ \mathrm{NTT}_{no\to bo,\beta}^{CT,\psi}(\boldsymbol{b})\right).$$

A high-level description of incomplete FFT trick over $\mathbb{Z}_q[x]/(x^n + 1)$

$$\mathbb{Z}_q[x]/(x^n + 1) \to (\mathbb{Z}_q[x]/(x^{2^\beta} - y))[y]/(y^{\frac{n}{2^\beta}} + 1),$$

along with rewriting $\boldsymbol{a} \in \mathbb{Z}_q[x]/(x^n + 1)$ as $\boldsymbol{a} = \sum_{i=0}^{n/2\beta-1} \tilde{a}_i y^i$, where $y = x^{2^\beta}$ and $\tilde{a}_i = \sum_{j=0}^{2^\beta-1} a_{2^\beta \cdot i+j} x^j \in \mathbb{Z}_q[x]/(x^{2^\beta} - y)$.

Thus, $\boldsymbol{a}$ can be seen as a polynomial of degree $(n/2^\beta - 1)$ with respect to y.

FFT trick will map

$$(\mathbb{Z}_q[x]/(x^{2^\beta} - y))[y]/(y^{\frac{n}{2^\beta}} + 1) \cong \prod_{i=0}^{n/2^\beta - 1} (\mathbb{Z}_q[x]/(x^{2^\beta} - y))[y]/(y - \psi_{2n/2^\beta}^{2\mathrm{brv}_{n/2^\beta}(i)+1}).$$

Its forward transform (resp., inverse transform) is treated as radix-2 $n/2^\beta$-point full NWC-based $\mathrm{NTT}_{no\to bo}^{CT,\psi}$ (resp., $\mathrm{INTT}_{bo\to no}^{GS,\psi^{-1}}$) with respect to y.

And the point-wise multiplication is performed in $(\mathbb{Z}_q[x]/(x^{2^\beta} - y))[y]/(y - \psi_{2n/2^\beta}^{2\mathrm{brv}_{n/2^\beta}(i)+1})$.

Method based on incomplete FFT trick over $\mathbb{Z}_q[x]/(x^n-1)$

The method based on incomplete FFT trick over$\mathbb{Z}_q[x]/(x^n-1)$ achieves its CRT map as follows.

$$\mathbb{Z}_q[x]/(x^n-1) \cong \prod_{i=0}^{n/2^\beta-1} \mathbb{Z}_q[x]/(x^{2^\beta} - \omega_{n/2^\beta}^{\mathrm{brv}_{n/2^\beta}(i)}).$$

where $\beta = 0, 1, \ldots, \log n - 1$.

Similarly, denote by $\mathrm{NTT}^{CT}_{no\to bo,\beta}$ and $\mathrm{INTT}^{GS}_{bo\to no,\beta}$ the forward and the inverse transform.

The restriction on q can be weakened to $q \equiv 1 \pmod{\frac{n}{2^\beta}}$.

Its way to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n-1)$ is

$$\boldsymbol{c} = \mathrm{INTT}^{GS}_{bo\to no,\beta}\left(\mathrm{NTT}^{CT}_{no\to bo,\beta}(\boldsymbol{a}) \circ \mathrm{NTT}^{CT}_{no\to bo,\beta}(\boldsymbol{b})\right).$$

Its high-level description can be written as

$$\mathbb{Z}_q[x]/(x^n-1) \to (\mathbb{Z}_q[x]/(x^{2^\beta} - y))[y]/(y^{\frac{n}{2^\beta}} - 1),$$

followed by CRT isomorphism.

$$(\mathbb{Z}_q[x]/(x^{2^\beta} - y))[y]/(y^{\frac{n}{2^\beta}} - 1) \cong \prod_{i=0}^{n/2^\beta-1} (\mathbb{Z}_q[x]/(x^{2^\beta} - y))[y]/(y - \omega_{n/2^\beta}^{\mathrm{brv}_{n/2^\beta}(i)}).$$

4.2 Method Based on Splitting Polynomial Ring

The computing strategies of Pt-NTT, K-NTT, and H-NTT are similarly dependent on splitting the initial polynomial ring, based on which they are classified as the method based on splitting the polynomial ring. And their basic idea can be traced back to Nussbaumer's trick. Following the description of Nussbaumer's trick, they can essentially be described by the following isomorphism. Let α be a non-negative integer.

$$\Psi_\alpha : \mathbb{Z}_q[x]/(x^n \pm 1) \cong \left(\mathbb{Z}_q[y]/(y^{\frac{n}{2^\alpha}} \pm 1)\right)[x]/(x^{2^\alpha} - y)$$

$$a = \sum_{i=0}^{n-1} a_i x^i \mapsto \Psi_\alpha(a) = \sum_{i=0}^{2^\alpha-1} \left(\sum_{j=0}^{\frac{n}{2^\alpha}-1} a_{2^\alpha \cdot j + i} y^j \right) x^i.$$

Obviously, the isomorphism Ψ_α and its inverse Ψ_α^{-1} only perform simple reordering of the polynomial coefficients. Ψ_α is the identity mapping if $\alpha = 0$.

Briefly speaking, the general form of α-round method based on splitting polynomial ring to compute $c = a \cdot b \in \mathbb{Z}_q[x]/(x^n \pm 1)$ mainly contains the following three steps, where n is a power of two and q is a prime number (more details about q can be seen below).

- Step 1, Splitting.

  The polynomials a and b are split by Ψ_α into

  $$\Psi_\alpha(a) = \sum_{i=0}^{2^\alpha-1} \tilde{a}_i \cdot x^i, \quad \Psi_\alpha(b) = \sum_{i=0}^{2^\alpha-1} \tilde{b}_i \cdot x^i \in \left(\mathbb{Z}_q[y]/(y^{\frac{n}{2^\alpha}} \pm 1)\right)[x]/(x^{2^\alpha} - y),$$

  where $y = x^{2^\alpha}$, and

  $$\tilde{a}_i = \sum_{j=0}^{\frac{n}{2^\alpha}-1} a_{2^\alpha \cdot j + i} y^j, \quad \tilde{b}_i = \sum_{j=0}^{\frac{n}{2^\alpha}-1} b_{2^\alpha \cdot j + i} y^j \in \mathbb{Z}_q[y]/(y^{\frac{n}{2^\alpha}} \pm 1).$$

- Step 2, Multiplication.

  The product of $\Psi_\alpha(a)$ and $\Psi_\alpha(b)$ is obtained by $(\sum_{i=0}^{2^\alpha-1}\tilde{a}_i\cdot x^i)(\sum_{i=0}^{2^\alpha-1}\tilde{b}_i\cdot x^i) \pmod{x^{2^\alpha}-y}$, which means that one need to compute $\tilde{c}_i \in \mathbb{Z}_q[y]/(y^{n/2\alpha}\pm 1)$ for $i=0,1,\ldots,2^\alpha-1$ as follows.

$$\tilde{c}_i = \sum_{l=0}^{i}\tilde{a}_l\cdot\tilde{b}_{i-l} + \sum_{l=i+1}^{2^\alpha-1} y\cdot\tilde{a}_l\cdot\tilde{b}_{2^\alpha+i-l} \in \mathbb{Z}_q[y]/(y^{n/2\alpha}\pm 1).$$

- Step 3, Gatheration.

  Gather all the $\tilde{c}_i$ by Ψ_α^{-1}, and obtain $c=\Psi_\alpha^{-1}\left(\sum_{i=0}^{2^\alpha-1}\tilde{c}_i\cdot x^i\right)$.

Step 1 and Step 3 are simple and easy.

Essentially, Ψ_α transforms NTT/INTT over $\mathbb{Z}_q[x]/(x^n\pm 1)$ into those over $\mathbb{Z}_q[y]/(y^{n/2\alpha}\pm 1)$ which requires only $n/2^\alpha$-point NTT/INTT with arbitrary appropriate modulus q, but the point-wise multiplication needs to be adapted to NTT/INTT.

There are three variants based on the method based on splitting polynomial ring, including Pt-NTT, K-NTT, H-NTT. The main difference between them is that they use different skills and NTTs to compute $\tilde{c}_i$ of Step 2.

6.2.1 Pt-NTT

Preprocess-then-NTT (Pt-NTT) improves formula as follows.

$$\begin{aligned}\tilde{c}_i &= \sum_{l=0}^{i}\tilde{a}_l\cdot\tilde{b}_{i-l} + \sum_{l=i+1}^{2^\alpha-1}\vec{a}_l\cdot\tilde{b}_{2^\alpha+i-l}\\ &= \mathrm{INTT}\left(\sum_{l=0}^{i}\mathrm{NTT}(\tilde{a}_l)\circ\mathrm{NTT}(\tilde{b}_{i-l}) + \sum_{l=i+1}^{2^\alpha-1}\mathrm{NTT}(\vec{a}_l)\circ\mathrm{NTT}(\tilde{b}_{2^\alpha+i-l})\right),\end{aligned}$$

where "$\circ$" is the corresponding point-wise multiplication, and for $\mathbb{Z}_q[y]/(y^{n/2\alpha}+1)$, there is

$$\tilde{\tilde{a}}_l = y\cdot\tilde{a}_l = -a_{n-2^\alpha+l} + \sum_{j=0}^{n/2\alpha-2} a_{2^\alpha\cdot j+l}y^{j+1} \in \mathbb{Z}_q[y]/(y^{n/2\alpha}+1),$$

or, for $\mathbb{Z}_q[y]/(y^{n/2\alpha}-1)$, there is

$$\tilde{\tilde{a}}_l = y\cdot\tilde{a}_l = a_{n-2^\alpha+l} + \sum_{j=0}^{n/2\alpha-2} a_{2^\alpha\cdot j+l}y^{j+1} \in \mathbb{Z}_q[y]/(y^{n/2\alpha}-1).$$

Here, Pt-NTT uses $n/2^\alpha$-point full NWC-based NTT/INTT over $\mathbb{Z}_q[y]/(y^{n/2\alpha}+1)$, or $n/2^\alpha$-point full CC-based NTT/INTT over $\mathbb{Z}_q[y]/(y^{n/2\alpha}-1)$.

6.2.2 K-NTT

Later, Karatsuba-NTT (K-NTT) is proposed based on Pt-NTT, equipping with one-iteration Karatsuba algorithm. Its Step 2 is given as

$$\begin{aligned}\tilde{c}_i &= \sum_{l=0}^{i}\tilde{a}_l\cdot\tilde{b}_{i-l} + \sum_{l=i+1}^{2^\alpha-1} y\cdot\tilde{a}_l\cdot\tilde{b}_{2^\alpha+i-l}\\ &= \mathrm{INTT}\left(\sum_{l=0}^{i}\mathrm{NTT}(\tilde{a}_l)\circ\mathrm{NTT}(\tilde{b}_{i-l}) + \sum_{l=i+1}^{2^\alpha-1}\mathrm{NTT}(y)\circ\mathrm{NTT}(\tilde{a}_l)\circ\mathrm{NTT}(\tilde{b}_{2^\alpha+i-l})\right).\end{aligned}$$

Here, K-NTT uses $n/2^{\alpha}$-point full NWC-based NTT/INTT over $\mathbb{Z}_q[y]/(y^{n/2\alpha}+1)$, or $n/2^{\alpha}$-point full CC-based NTT/INTT over $\mathbb{Z}_q[y]/(y^{n/2\alpha}-1)$.

Since y has been known, NTT(y) can be computed and stored offline in advance. In Step 2, one-iteration Karatsuba algorithm is used in such a manner.

First compute and store $\mathrm{NTT}(\tilde{a}_i) \circ \mathrm{NTT}(\tilde{b}_i)$ for any $i = j$,

and then

$$\begin{aligned}
&\mathrm{NTT}(\tilde{a}_i) \circ \mathrm{NTT}(\tilde{b}_j) + \mathrm{NTT}(\tilde{a}_j) \circ \mathrm{NTT}(\tilde{b}_i) \\
&= (\mathrm{NTT}(\tilde{a}_i) + \mathrm{NTT}(\tilde{a}_j)) \circ (\mathrm{NTT}(b_i) + \mathrm{NTT}(b_j)) - \mathrm{NTT}(\tilde{a}_i) \circ \mathrm{NTT}(b_i) - \mathrm{NTT}(\tilde{a}_j) \circ \mathrm{NTT}(b_j)
\end{aligned}$$

for any $i \neq j$.

6.2.3 H-NTT

Furthermore, Hybrid-NTT (H-NTT) is proposed as an improved K-NTT and a new variant of NTT, by applying truncated-NTT in Step 2 and one-iteration Karatsuba algorithm in its point-wise multiplication.

H-NTT uses truncated-NTT with β levels cropped, instead of those full NTT in K-NTT. Its Step 2 is given as

$$\begin{aligned}
\tilde{c}_i &= \sum_{l=0}^{i} \tilde{a}_l \cdot \tilde{b}_{i-l} + \sum_{l=i+1}^{2^{\alpha}-1} y \cdot \tilde{a}_l \cdot \tilde{b}_{2^{\alpha}+i-l} \\
&= \mathrm{INTT}_{\beta}\left(\sum_{l=0}^{i} \mathrm{NTT}_{\beta}(\tilde{a}_l) \circ \mathrm{NTT}_{\beta}(\tilde{b}_{i-l}) + \sum_{l=i+1}^{2^{\alpha}-1} \mathrm{NTT}_{\beta}(y) \circ \mathrm{NTT}_{\beta}(\tilde{a}_l) \circ \mathrm{NTT}_{\beta}(\tilde{b}_{2^{\alpha}+i-l}) \right),
\end{aligned}$$

where $\mathrm{NTT}_{\beta}/\mathrm{INTT}_{\beta}$ means NWC-based $\mathrm{NTT}^{CT,\psi}_{no\to bo,\beta}/\mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no,\beta}$ over $\mathbb{Z}_q[y]/(y^{n/2\alpha}+1)$,

or CC-based $\mathrm{NTT}^{CT}_{no\to bo,\beta}/\mathrm{INTT}^{GS}_{bo\to no,\beta}$ over $\mathbb{Z}_q[y]/(y^{n/2\alpha}-1)$.

One-iteration Karatsuba algorithm is also used in its point-wise multiplication.

For example, to compute $(\sum_{i=0}^{2^{\beta}-1} \hat{a}_i x^i)(\sum_{i=0}^{2^{\beta}-1} \hat{b}_i x^i) \pmod{x^{2^{\beta}} - \psi}$, one can compute $\hat{a}_i\hat{b}_j$ for any $i = j$ first and then compute $\hat{a}_i\hat{b}_j + \hat{a}_j\hat{b}_i = (\hat{a}_i + \hat{a}_j)(\hat{b}_i + \hat{b}_j) - \hat{a}_i\hat{b}_i - \hat{a}_j\hat{b}_j$ for any $i \neq j$.

6.2.4 Comparisons and discussions

Based on the high-level description of incomplete FFT trick, it shares some similarities with the method based on splitting polynomial ring.

In fact, there is an isomorphism between $(\mathbb{Z}_q[x]/(x^{2^{\beta}} - y))[y]/(y^{n/2\beta} \pm 1)$ and $(\mathbb{Z}_q[y]/(y^{n/2\alpha} \pm 1))[x]/(x^{2^{\alpha}} - y)$ for any $\alpha = \beta$. And they have been proved computationally equivalent, which implies that their efficiencies are the same theoretically.

These two methods can expand the value range of modulus q, because q can only satisfy $q \equiv 1 \pmod{2n/2^{\alpha+\beta}}$ for some α, β, instead of $q \equiv 1 \pmod{2n}$ when n is fixed, for NWC-based NTT; besides, q can only satisfy $q \equiv 1 \pmod{n/2\alpha+\beta}$ for some α, β, instead of $q \equiv 1 \pmod{n}$ when n is fixed, for CC-based NTT.

However, the limitations on q can not be ignored, since q must be an NTT-friendly prime such that $\mathbb{Z}_q$ is a finite field and $x^n \pm 1$ can be split into polynomials of small degree over $\mathbb{Z}_q$.

In addition, as for the method based on splitting polynomial ring, the polynomial operations over $\mathbb{Z}_q[x]/(x^n \pm 1)$ can be transformed into those over a smaller ring $\mathbb{Z}_q[y]/(y^{n/2\alpha} \pm 1)$. It leads to more modular implementation, since NTT over $\mathbb{Z}_q[y]/(y^{n/2\alpha} \pm 1)$ can be implemented as a "black box", and be re-called when required in Step 2.

Both Pt-NTT and K-NTT use $n/2^\alpha$-point full NTT in their Step 2. It requires that the primitive $n/2^{\alpha-1}$-th root of unity exits for NWC-based NTT, or the primitive $n/2^\alpha$-th root of unity exits for CC-based NTT.

Therefore, the parameter condition of Pt-NTT and K-NTT can be weakened to $q \equiv 1 \pmod{2n/2^\alpha}$ for the initial ring $\mathbb{Z}_q[x]/(x^n + 1)$, or $q \equiv 1 \pmod{n/2\alpha}$ for the initial ring $\mathbb{Z}_q[x]/(x^n - 1)$. H-NTT can further weaken the parameter condition to $q \equiv 1 \pmod{2n/2^{\alpha+\beta}}$ for the ring $\mathbb{Z}_q[x]/(x^n + 1)$ by using truncated-NTT, or $q \equiv 1 \pmod{n/2\alpha + \beta}$ for the ring $\mathbb{Z}_q[x]/(x^n - 1)$.

By comparison, it can be seen that α-round K-NTT and truncated-NTT are the special cases of H-NTT when $\beta = 0$ and $\alpha = 0$ respectively.

More specially, $\mathrm{NTT}^{CT,\psi}_{no\to bo}$ and $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}$ are the cases of H-NTT over $\mathbb{Z}_q[x]/(x^n + 1)$ when $\alpha = 0, \beta = 0$, and $\mathrm{NTT}^{CT}_{no\to bo}$ and $\mathrm{INTT}^{GS}_{bo\to no}$ are the cases of H-NTT over $\mathbb{Z}_q[x]/(x^n - 1)$ when $\alpha = 0, \beta = 0$.

4.3 Method Based on Large Modulus

Here we consider $\mathbb{Z}_q[x]/(x^n \pm 1)$, where n is a power of two and the modulus q is an NTT-unfriendly prime, but q can be actually any positive integer.

The process of $c = a \cdot b \in \mathbb{Z}_q[x]/(x^n \pm 1)$ is divided into two steps.

- Step 1.

  Compute $c' = a \cdot b \in \mathbb{Z}_N[x]/(x^n \pm 1)$, where N is a positive integer and larger than the maximum absolute value of the coefficients during the computation over $\mathbb{Z}$.

- Step 2.

  One can recover the result in $\mathbb{Z}_q[x]/(x^n \pm 1)$ through reduction module q, i.e., $c = c' \bmod q$.

It is called the method based on large modulus in this paper, because its key step is to select a large enough modulus N in Step 1.

Obviously, N could be chosen such that $N > nq^2$. When N is large enough, the product of a and b in $\mathbb{Z}[x]/(x^n \pm 1)$ is identical to that in $\mathbb{Z}_N[x]/(x^n \pm 1)$.

In order to apply NTTs, we consider two sub-cases of the values of N. One is that N is an NTT-friendly prime. The other is that N is the product of some NTT-friendly primes. When N is the product of some NTT-friendly primes, i.e., $N = \prod_{i=1}^{l} q_i$ where each q_i is an NTT-friendly prime, in this case, it can further be classified into two sub-methods.

One is the method based on residue number system (RNS). The other is the method based on composite-modulus ring.

6.3.1 Method based on NTT-friendly large prime.

When N is an NTT-friendly prime, NTT can be performed in $\mathbb{Z}_N[x]/(x^n \pm 1)$ directly in Step 1.

Notice that if N is set sufficiently large and NTT-friendly, this method always works regardless of the value of the original modulus q.

For example, if N is a prime satisfying $N > nq^2$ and $N \equiv 1 \pmod{2n}$, n-point full NWC-based NTT of modulus N can always be used over $\mathbb{Z}_N[x]/(x^n + 1)$. Besides, if N is a prime satisfying $N > nq^2$ and $N \equiv 1 \pmod{n}$, n-point full CC-based NTT of modulus N can always be used over $\mathbb{Z}_N[x]/(x^n - 1)$.

6.3.2 Method based on residue number system.

Residue number system (RNS) is widely used in the context of homomorphic encryption for computing NTTs over many primes.

Based on RNS, negative wrapped convolution with a modulus that is the product of some primes, can be transformed into ones with smaller moduli by Chinese Remainder Theorem, that is

$$\mathbb{Z}_N[x]/(x^n \pm 1) \cong \prod_{i=1}^{l} \mathbb{Z}_{q_i}[x]/(x^n \pm 1).$$

Denote by c_i the product of a and b in $\mathbb{Z}_{q_i}[x]/(x^n \pm 1)$ for $i = 1, \ldots, l$.

After using NTTs to compute $c_i, i = 1, \ldots, l$, the original product c in $\mathbb{Z}_N[x]/(x^n \pm 1)$ can be recovered from $c_i, i = 1, \ldots, l$, by Chinese Remainder Theorem in number theoretic.

6.3.3 Method based on composite-modulus ring.

The NTT can be performed over a polynomial ring with a composite modulus directly.

Specifically, generalizes the concepts of NTT from a finite field to an integer ring, the basic idea of which was first developed in terms of FFT.

Consider $\mathbb{Z}_N[x]/(x^n + 1)$, where $N = \prod_{i=1}^{l} q_i$ and each q_i is NTT-friendly prime. If $2n \mid \gcd(q_1 - 1, \ldots, q_l - 1)$, there exits a principal $2n$-th root of unity ψ_{2n} in $\mathbb{Z}_N$.

ψ_{2n} is a principle $2n$-th root of unity in $\mathbb{Z}_N$, iff $(\psi_{2n} \bmod q_i)$ is a principle $2n$-th root of unity in $\mathbb{Z}_{q_i}$ for any i.

$$\mathbb{Z}_N[x]/(x^n + 1) \cong \prod_{i=0}^{n-1} \mathbb{Z}_N[x]/(x - \psi_{2n}^{2brv_n(i)+1}).$$

Its forward transform and inverse transform are illustrated as $\mathrm{NTT}_{no \to bo}^{CT,\psi}$ and $\mathrm{INTT}_{bo \to no}^{GS,\psi^{-1}}$ over $\mathbb{Z}_N[x]/(x^n+1)$.

The CRT map of truncated-NTT with β levels cropped is as follows.

$$\mathbb{Z}_N[x]/(x^n + 1) \cong \prod_{i=0}^{n/2^\beta - 1} \mathbb{Z}_N[x]/(x^{2^\beta} - \psi_{2n/2^\beta}^{2brv_{n/2^\beta}(i)+1}),$$

where $\psi_{2n/2^\beta}$ is a principal $2n/2^\beta$-th root of unity in $\mathbb{Z}_N$.

Its forward transform and inverse transform are illustrated as $\mathrm{NTT}_{no \to bo,\beta}^{CT,\psi}$ and $\mathrm{INTT}_{bo \to no,\beta}^{GS,\psi^{-1}}$ over $\mathbb{Z}_N[x]/(x^n+1)$, respectively.

As for $\mathbb{Z}_N[x]/(x^n-1)$, where $N=\prod_{i=1}^{l} q_i$ and each q_i is NTT-friendly prime.

If $n \mid \gcd(q_1-1,\ldots,q_l-1)$, there exits a principal n-th root of unity ω_n in $\mathbb{Z}_N$.

ω_n is a principle n-th root of unity in $\mathbb{Z}_N$, iff $(\omega_n \bmod q_i)$ is a principle n-th root of unity in $\mathbb{Z}_{q_i}$ for any i.

The CRT map of full-mapping FFT trick and truncated-NTT with β levels cropped over $\mathbb{Z}_N[x]/(x^n-1)$ can be derived similarly to those over $\mathbb{Z}_N[x]/(x^n+1)$.

6.3.4 Comparisons and discussions.

Compare the method based on large modulus and those based on incomplete FFT trick/splitting polynomial ring.

The three sub-methods of the method based on large modulus are valid for any original modulus q (including NTT-friendly ones), and can completely remove the restriction on q. But, their shortcomings are also obvious.

Specifically, the method based on incomplete FFT trick and based on splitting polynomial ring still choose original q as the modulus. But, the modulus N used in the method based on NTT-friendly large prime and method based on composite-modulus ring, is much larger than original modulus q.

For examples, N could be chosen as nq^2. Although N be smaller if one of the multiplicands is small, it will still be several orders of magnitude larger than q.

It causes that the storage of coefficients and the computing-resource consume will be more than the cases of q during the computation. Besides, as for the method based on residue number system, there are more than one NTT computation needed to be computed. Traditionally, it is more time-consuming and resource-consuming, because full NWC-based NTT, the method based on incomplete FFT trick and the method based on splitting polynomial ring only need one NTT computation.

Therefore, if q is an NTT-friendly prime number, the methods based on incomplete FFT trick and based on splitting polynomial ring are recommended strongly for an efficient implementation, instead of the method based on large modulus. But, when q is an NTT-unfriendly prime number, one can turn to the method based on large modulus.

Note that, there are some connections between the method based on residue number system and composite-modulus ring.

Here review $\mathbb{Z}_N[x]/(x^n \pm 1)$. Both of them could choose $N=\prod_{i=1}^{l} q_i$ where each q_i is NTT-friendly prime. If one split N via CRT and keep $x^n \pm 1$ unchanged, it implies the method based on residue number system.

If one split $x^n \pm 1$ via CRT and keep N unchanged, it implies the method based on composite-modulus ring.

Therefore, these two methods are derived from different splitting forms of moduli via CRT.

5 CHOOSING NUMBER THEORETIC TRANSFORM FOR GIVEN RINGS

This section introduces how to choose an appropriate NTT for the given polynomial ring, mainly classifying the rings into three categories for the convenience of understanding.

- $\mathbb{Z}_q[x]/(x^n \pm 1)$ with respect to power-of-two n
- $\mathbb{Z}_q[x]/(x^n \pm 1)$ with respect to non-power-of-two n
- $\mathbb{Z}_q[x]/(\phi(x))$ with respect to general $\phi(x)$ of degree n.

Mainly focus on the special rings of the form $\mathbb{Z}_q[x]/(x^n \pm 1)$ in the first two categories, and then extend the results to the general rings of the form $\mathbb{Z}_q[x]/(\phi(x))$ with respect to general $\phi(x)$ of degree n.

5.1 $\mathbb{Z}_q[x]/(x^n \pm 1)$ with Respect to Power-of-two n

NTT-friendly q

In this case, q is an NTT-friendly prime of the form $q = q' \cdot 2^e + 1$.

Furthermore, classify it into two sub-cases, the first sub-case is that q can lead to a full NTT.

The another sub-case is that q can not lead to a full NTT.

First consider the first sub-case of n being a power of two and q leading to a full NTT.

As for $\mathbb{Z}_q[x]/(x^n+1)$, there exits $q \equiv 1 \pmod{2n}$, then full NWC-based NTT (e.g., $\text{NTT}^{CT,\psi}_{no\to bo}$ and $\text{INTT}^{GS,\psi^{-1}}_{bo\to no}$) can be used to multiply two polynomials in $\mathbb{Z}_q[x]/(x^n + 1)$.

Restate that one can compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n + 1)$ by

$$\boldsymbol{c} = \text{INTT}^{GS,\psi^{-1}}_{bo\to no}\left(\text{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{a}) \circ \text{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{b})\right).$$

As for $\mathbb{Z}_q[x]/(x^n - 1)$, there exits $q \equiv 1 \pmod{n}$, then full CC-based NTT (e.g., $\text{NTT}^{CT}_{no\to bo}$ and $\text{INTT}^{GS}_{bo\to no}$) can be used to multiply two polynomials in $\mathbb{Z}_q[x]/(x^n - 1)$.

Restate that one can compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n - 1)$ by

$$\boldsymbol{c} = \text{INTT}^{GS}_{bo\to no}\left(\text{NTT}^{CT}_{no\to bo}(\boldsymbol{a}) \circ \text{NTT}^{CT}_{no\to bo}(\boldsymbol{b})\right).$$

Then consider the another sub-case of n being a power of two and q not leading to a full NTT.

It means that, for $\mathbb{Z}_q[x]/(x^n + 1)$, q does not satisfy $q \equiv 1 \pmod{2n}$,

or, for $\mathbb{Z}_q[x]/(x^n - 1)$, q does not satisfy $q \equiv 1 \pmod{n}$.

In this sub-case, the method based on incomplete FFT trick and the method splitting polynomial ring are highly recommended.

For example, as for $\mathbb{Z}_q[x]/(x^n + 1)$, if q only satisfies $q \equiv 1 \pmod{2n/2^\alpha}$ (resp., $q \equiv 1 \pmod{2n/2^\beta}$), then one can use α-round method based on splitting polynomial ring (resp., method based on incomplete FFT trick with β levels cropped) can be used to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n + 1)$.

Besides, as for $\mathbb{Z}_q[x]/(x^n - 1)$, if q only satisfies $q \equiv 1 \pmod{n/2\alpha}$ (resp., $q \equiv 1 \pmod{n/2\beta}$), then one can use α-round method based on splitting polynomial ring (resp., method based on incomplete FFT trick with β levels cropped) can be used to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n - 1)$.

Furthermore, if q only satisfies $q \equiv 1 \pmod{2n/2^{\alpha+\beta}}$ for $\mathbb{Z}_q[x]/(x^n + 1)$, one can use H-NTT with α-round splitting and β levels cropped;

similarly, if q only satisfies $q \equiv 1 \pmod{n/2\alpha + \beta}$ for $\mathbb{Z}_q[x]/(x^n - 1)$, one can use H-NTT with α-round splitting and β levels cropped.

NTT-unfriendly q

In this case, q is an NTT-unfriendly prime.

In order to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n \pm 1)$, one can use the method based on large modulus.

Firstly, compute $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_N[x]/(x^n \pm 1)$, where N is a positive integer and larger than the maximum absolute value of the coefficients during the computation over $\mathbb{Z}$, and then compute $\boldsymbol{c} = \boldsymbol{c}' \bmod q$.

To compute $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_N[x]/(x^n \pm 1)$, one can use the method based on NTT-friendly large prime, the method based on residue number system or the method based on composite-modulus ring.

5.2 $\mathbb{Z}_q[x]/(x^n \pm 1)$ with Respect to Non-power-of-two n

Consider the case of non-power-of-two n, but actually n can be a general integer.

The essential idea about the computation of $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^n \pm 1)$ is described through two steps as follows.

1. Step 1.

   Compute $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^{n'} \pm 1)$, where n' is a larger integer and $n' \geq 2n$.

2. Step 2.

   One can recover the result in $\mathbb{Z}_q[x]/(x^n \pm 1)$ through reduction modulo $x^{n'} \pm 1$, i.e., $\boldsymbol{c} = \boldsymbol{c}' \bmod x^{n'} \pm 1$.

   As for the ring $\mathbb{Z}_q[x]/(x^{n'} \pm 1)$

   (a) The first case is that n' is a power of two, which is denoted by 2^k.

   (b) The other case is that n' is of the form $h \cdot 2^k$, where h is an odd number.

Power-of-two n'

As for the first case, the computation is transformed into that with respect to $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^{n'} \pm 1)$, where n' is a power of two.

Use Good's trick

As for the other case, the computation is transformed into that with respect to $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(x^{n'} \pm 1)$, where $n' = h \cdot 2^k$ and h is an odd number.

One can use Good's trick to compute polynomial multiplication over $\mathbb{Z}_q[x]/(x^{h \cdot 2^k} - 1)$. In this paper, Good's trick is recommended, since there are more freedom to select an odd number h and power-of-two 2^k. If one expand the length to power-of-two n', it is inconvenient to find a suitable polynomial of some particular degree up to the next power of two.

5.3 $\mathbb{Z}_q[x]/(\phi(x))$ with Respect to General $\phi(x)$ of Degree n

Here we consider the more general ring $\mathbb{Z}_q[x]/(\phi(x))$ where $\phi(x)$ is an arbitrary polynomial of degree n. In order to apply fast NTT algorithm, there are complex but efficient methods.

The computation of $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(\phi(x))$ can be described through two steps as follows.

1. Step 1.

   Compute $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_N[x]/(x^{n'} \pm 1)$, where N is a positive integer and larger than the maximum absolute value of the coefficients during the computation over $\mathbb{Z}$, n' is a larger integer and $n' \geq 2n$.

2. Step 2.

   And then compute $\boldsymbol{c} = (\boldsymbol{c}' \bmod \phi(x)) \bmod q$.

Therefore, the computation of $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_q[x]/(\phi(x))$ is transformed into that of $\boldsymbol{c}' = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathbb{Z}_N[x]/(x^{n'} \pm 1)$.

6 NUMBER THEORETIC TRANSFORM IN NIST PQC

NIST PQC Round 3 candidates' schemes mainly focus on their NTT-based implementations over their underlying rings $\mathbb{Z}_q[x]/(\phi(x))$, where $\mathbb{Z}_q[x]/(\phi(x))$ is instantiated as in Table.

Schemes	n	q	Rings	Types	Methods & Algorithms
Kyber Round 1 (*조정 전 파라미터)	256	7681	$\mathbb{Z}_q[x]/(x^n+1)$	NTT-friendly $q \equiv 1 \pmod{2n}$	n-point full NWC-based NTT
Kyber Round 2, Kyber Round 3	256	3329	$\mathbb{Z}_q[x]/(x^n+1)$	NTT-friendly $q \equiv 1 \pmod{n}$	Incomplete FFT trick, Splitting polynomial ring
Dilithium Round 3	256	8380417	$\mathbb{Z}_q[x]/(x^n+1)$	NTT-friendly $q \equiv 1 \pmod{2n}$	n-point full NWC-based NTT

Notice that not all the schemes can directly use full NTT to multiply polynomials. For example, full NWC-based NTT over $\mathbb{Z}_q[x]/(x^n+1)$ further requires that the prime q satisfies $q \equiv 1 \pmod{2n}$. Only Dilithium and Falcon can meet the situation. Besides, full NTT can not be directly used in those lattice-based schemes with power-of-two moduli, such as Saber and NTRU.

6.1 Kyber

Kyber is an IND-CCA secure MLWE-based KEM from the lattice-based cryptography suite called "Cryptographic Suite for Algebraic Lattices (CRYSTALS)".

The Kyber submission in NIST PQC Round 1, named Kyber Round 1, uses $n = 256$ and $q = 7681$ which satisfies $q \equiv 1 \pmod{2n}$.

Its way to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathcal{R}_q$ in C reference implementation is elaborated as

$$c = \psi^{-1} \circ \mathrm{INTT}^{GS}_{bo \to no} \left(\mathrm{NTT}^{CT,\psi}_{no \to bo}(\boldsymbol{a}) \circ \mathrm{NTT}^{CT,\psi}_{no \to bo}(\boldsymbol{b}) \right),$$

where the inverse transform is $\psi^{-1} \circ \mathrm{INTT}^{GS}_{bo \to no}$, the output of which is the same as that of $\mathrm{INTT}^{GS,\psi^{-1}}_{bo \to no}$.

As for its AVX2 optimized implementation, Kyber Round 1 uses $\mathrm{NTT}^{CT,\psi}_{no\to bo}$ and $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}$, and the computation is restated as

$$c = \mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}\left(\mathrm{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{a}) \circ \mathrm{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{b})\right).$$

The Kyber submissions in the second and third round of NIST PQC competition use a smaller prime number $q = 3329$ which no longer satisfies $q \equiv 1 \pmod{2n}$, but $q \equiv 1 \pmod{n}$.

Kyber Round 2 and Round 3 use truncated-NTT with one level cropped: $\mathrm{NTT}^{CT,\psi}_{no\to bo,\beta=1}$ and $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no,\beta=1}$.

The point-wise multiplication is treated as the corresponding polynomial multiplications in $\mathbb{Z}_q[x]/(x^2-\omega_n^{2brv_{n/2}(i)+1})$.

Its way to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathcal{R}_q$ is restated as

$$c = \mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no,\beta=1}\left(\mathrm{NTT}^{CT,\psi}_{no\to bo,\beta=1}(\boldsymbol{a}) \circ \mathrm{NTT}^{CT,\psi}_{no\to bo,\beta=1}(\boldsymbol{b})\right).$$

Further, Liang et al. improve its truncated-NTT by using H-NTT with $\alpha = 0$, which can be seen as $\mathrm{NTT}^{CT,\psi}_{no\to bo,\beta=1}$ and $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no,\beta=1}$ with one-iteration Karatsuba algorithm in its point-wise multiplication.

Different from truncated-NTT used in Kyber Round 2 and Round 3, they used 1-round Pt-NTT with $n/2$-point full NWC-based NTT for "small-Kyber". But, their implementation had a slightly worse performance than the initial one.

6.2 Dilithium

Dilithium is a signature scheme based on module lattice and is one of the algorithms from CRYSTALS.

Its parameter sets satisfy the condition $q \equiv 1 \pmod{2n}$ such that n-point full NWC-based $\mathrm{NTT}^{CT,\psi}_{no\to bo}$ and $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}$ can be utilized.

Its way to compute $\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \in \mathcal{R}_q$ is restated as

$$c = \mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}\left(\mathrm{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{a}) \circ \mathrm{NTT}^{CT,\psi}_{no\to bo}(\boldsymbol{b})\right).$$

C SOME SKILLS

Those lattice-based schemes fully utilizes the advantages of NTT to improve their efficiency.

As for MLWE-based schemes such as Kyber, Dilithium, etc, at a high level, their basic operations can be described as matrix-vector polynomial multiplication $\mathbf{A}^T\mathbf{r}$ and vector-vector polynomial multiplication $\mathbf{s}^T\mathbf{u}$, where $\mathbf{A} \in \mathcal{R}_q^{k\times k}$, $\mathbf{r}, \mathbf{s}, \mathbf{u} \in \mathcal{R}_q^{k\times 1}$.

Those schemes directly generate the public key term $\hat{\mathbf{A}}$ already in the NTT domain by rejection sampling, instead of generating $\mathbf{A}$ followed by applying forward transform on each element. It can save k^2 forward transforms.

The linearity of NTT can lead to $\mathrm{INTT}\left(\hat{\mathbf{A}}^T \circ \mathrm{NTT}(\mathbf{r})\right)$ where there are only k forward transforms and k inverse transforms.

Once the forward transform result $\hat{\mathbf{s}}$ is computed, $\hat{\mathbf{s}}$ can be stored or transmitted without any extra requirement of storage, for following use in multiple polynomial multiplications.

Using $\hat{\mathbf{s}}$ that is stored or transmitted in advance, one can compute $\mathbf{s}^T\mathbf{u}$ by $\mathrm{INTT}\left(\hat{\mathbf{s}}^T \circ \mathrm{NTT}(\mathbf{u})\right)$, within only k forward transforms and k inverse transforms.

As for MLWR-based schemes, the linearity of NTT is helpful in the NTT-based implementation of MLWR-based schemes such as Saber. The number of forward transforms and inverse transforms in Saber's matrix-vector multiplication $\mathbf{As}$ can be reduced from $2k^2$ and k^2 to $k^2 + k$ and k, respectively, while the number of inverse transforms $\mathbf{b}^T\mathbf{s}$ in vector-vector multiplication can be reduced from k to 1, where $\mathbf{A} \in \mathcal{R}_q^{k \times k}, \mathbf{b}, \mathbf{s} \in \mathcal{R}_q^{k \times 1}$.